THUẬT TOÁN

Gv: Đặng Hữu Nghị

Nội dung ôn tập thi Olympic tin học

- Các kĩ thuật lập trình cơ bản
- Nhập / xuất dữ liệu từ mảng 1 chiều, 2 chiều
- Các bài toán xử lý dữ liệu trên mảng 1 chiều, 2 chiều
- Chương trình con (Hàm) và ứng dụng
- Nhập / xuất dữ liệu từ tệp tin
- Xử lý xâu kí tự
- Một số thuật toán nâng cao
- V.v…

1. THUẬT TOÁN ĐỆ QUY

KHÁI NIỆM ĐỆ QUY

Có thể định nghĩa *hàm đệ quy* là hàm sẽ gọi đến chính nó .

Hàm đệ quy thường được viết theo thuật toán sau:

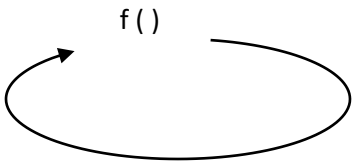
if (trường hợp cơ sở)
{
Trình bày cách giải bài toán (giả định đã có cách giải)
}
else
{
Gọi tới hàm đang lập với giá trị khác của tham số
}

PHÂN LOẠI ĐỆ QUY

- Đệ quy có thể được phân thành 4 loại sau:
  - Đệ quy tuyến tính
  - Đệ quy nhị phân
  - Đệ quy phi tuyến
  - Đệ quy hỗ tương

Đệ quy tuyến tính

- Hàm đệ quy tuyến tính là hàm mà trong thân hàm có duy nhất một lời gọi hàm gọi lại chính nó một cách tường minh.
- Thuật toán đệ quy tuyến tính:




```
<kiểu dữ liệu hàm> <tên hàm> (<danh sách tham số>)
{
        if (<điều kiện dừng>)
        {
                ...
                // Trả về giá trị hay kết thúc công việc
        }
        // Thực hiện một số công việc (nếu có)
        <tên hàm> (<danh sách tham số>) ;
        // Thực hiện một số công việc (nếu có)
}
```

Đệ quy tuyến tính

- Ví dụ: Tính n!

Ta nhận thấy *n!* có thể được tính theo công thức truy hồi như sau:

n! = 1 nếu n = 0 hoặc n = 1

n! = n . (n-1)! nếu n > 0

Dựa vào công thức trên ta có thể xây dựng hàm để tính n! một cách đệ quy như sau:

```
long giaithua ( int n)
{
    if ( n == 0 ||  n == 1)  return 1;
    else  return  ( n * giaithua ( n - 1 ) ) ;
}
```

Đệ quy nhị phân

- Hàm đệ quy nhị phân là hàm mà trong thân của nó có hai lời gọi hàm gọi lại chính nó một cách tường minh.
- Thuật toán đệ quy nhị phân:

Đệ quy nhị phân

```
<kiểu dữ liệu hàm> <tên hàm> (<danh sách tham số>)
{
        if (<điều kiện dừng>)
        {
                …
                //Trả về giá trị hay kết thúc công việc
        }
        //Thực hiện một số công việc (nếu có)
        <tên hàm> (<danh sách tham số>) ;   // Giải quyết vấn đề nhỏ hơn
        //Thực hiện một số công việc (nếu có)
        <tên hàm> (<danh sách tham số>);   // Giải quyết vấn đề còn lại
        //Thực hiện một số công việc nếu có
}
```

Đệ quy nhị phân

- Ví dụ 1: Dãy số Fibonaci được định nghĩa như sau:

  $f_0 = 0$, $f_1 = 1$,

  $f_n = f_{n-1} + f_{n-2}$ với $n > 1$.

Sau đây là chương trình xây dựng hàm đệ quy tìm số Fibonaci thứ n:

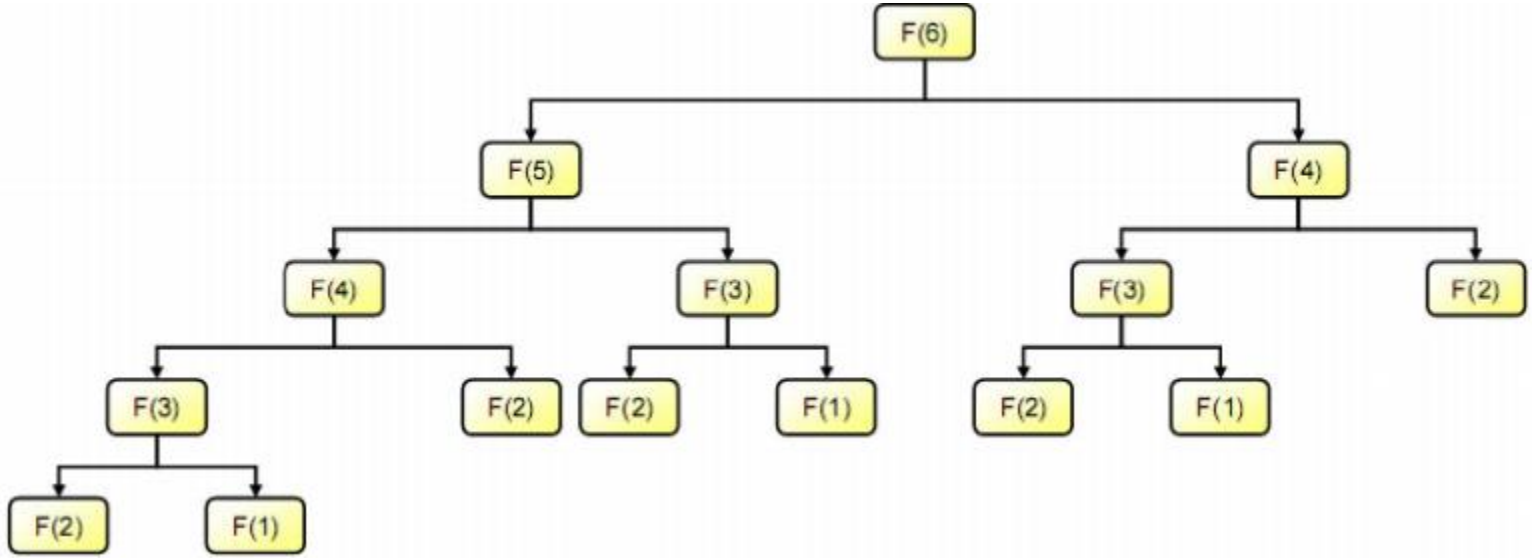
```
// Chuong trinh  tinh so Fibonaci thu n
#include <stdio.h>
#include <conio.h>

// ham de quy tinh so Fibonaci thu n
long fibonaci (long n)
{
   if (n == 0 || n == 1) return n;
   else return fibonaci(n-1)+ fibonaci(n-2);
}
```

Đệ quy nhị phân

```
void main()
{
   long n;
   clrscr();
   printf("Nhap vao n = ");
   scanf("%ld",&n);
   printf("So Fibonaci thu %ld la %ld",n,fibonaci(n));
   getch();
}
```

Đệ quy nhị phân



Đệ quy nhị phân

- Ví dụ 2: Cho dãy số nguyên $\{a_1, a_2, \ldots, a_n\}$ có thứ tự tăng dần, cho một số nguyên x, hãy tìm một phần tử trong dãy mà có giá trị bằng x.

Đệ quy nhị phân

- Thuật toán:
  - Do dãy số đã có thứ tự tăng dần nên chúng ta có thể áp dụng cách tìm kiếm theo phương pháp nhị phân. Ý tưởng của phương pháp này là: tại mỗi bước ta tiến hành so sánh x với phần tử nằm ở vị trí giữa của dãy để thu hẹp phạm vi tìm.
  - Gọi t là biên trái của dãy (ban đầu t = 0), p là biên phải của dãy (ban đầu p=n-1), g là vị trí ở giữa dãy (g = (t + p) / 2).
  - Thu hẹp phạm vi tìm dựa vào giá trị của phần tử ở giữa , có hai trường hợp:
    - Nếu x lớn hơn phần tử ở giữa thì x chỉ có thể xuất hiện ở bên phải vị trí này (từ g + 1 đến p).
    - Ngược lại nếu x nhỏ hơn phần tử ở giữa thì x chỉ có thể xuất hiện ở bên trái vị trí này (từ t đến g - 1).
  - Quá trình này thực hiện cho tới khi gặp phần tử có giá trị bằng x, hoặc đã xét hết các phần tử.

Đệ quy nhị phân

- Hàm đệ quy như sau:

```
int TimNhiPhan(int a[ ], int t, int p, int x)
{
        int g = (t + p) / 2 ;
        if ( t > p ) return -1 ;        // không có phần tử bằng x
        if(a[g] > x) return TimNhiPhan(a, t, g – 1, x);
        if(a[g] < x) return TimNhiPhan(a, g + 1, p, x);
        return m;               // Trả về vị trí tìm thấy
}
```

Đệ quy nhị phân

- Ví dụ 3: Bài toán tháp Hà nội – Có 3 cọc là A, B, C, cọc A chứa n đĩa lớn đến bé theo thứ tự từ dưới lên trên, hãy chuyển n đĩa này sang cọc C cũng theo thứ tự lớn dưới bé trên, sử dụng cọc trung gian B; chú ý khi di chuyển các đĩa phải tuân thủ nguyên tắc lớn dưới bé trên.
- Thuật toán đệ quy như sau:
  - Bước 1: Di chuyển n – 1 đĩa nhỏ hơn từ cọc A sang cọc B
  - Bước 2: Di chuyển đĩa còn lại từ cọc A sang cọc C
  - Bước 3: Di chuyển n – 1 điac nhỏ hơn từ cọc B sang cọc C

Đệ quy nhị phân

- Hàm đệ quy như sau:

```
void ThapHaNoi (int  n, char  A, char  B, char  C)
{
        if(n == 1) printf("\nDi chuyen tu coc %c den coc %c", A, C);
        else
                {
                        ThapHaNoi(n-1, A, C, B) ;
                        ThapHaNoi(1, A, B, C);
                        ThapHaNoi(n-1, B, A, C);
                }
}
```

Đệ quy phi tuyến

- Hàm đệ quy phi tuyến là hàm mà trong thân của hàm có lời gọi hàm gọi lại chính nó được đặt bên trong vòng lặp.
- Thuật toán đệ quy phi tuyến:

```
<kiểu dữ liệu hàm> <tên hàm> (<danh sách tham số>)
{
    for(int  i = 1; i <= n; i++)
      {
          //Thực hiện một số công việc (nếu có)
          if (<điều kiện dừng>){
                    …
                    //Trả về giá trị hay kết thúc công việc
        }  else
        {           //Thực hiện một số công việc (nếu có)
                    <tên hàm> (<danh sách tham số>) ;
                    //Thực hiện một số công việc (nếu có)
        }}}
```

Đệ quy phi tuyến

- Ví dụ:Tính số hạng thứ n của dãy số $x_1, x_2, \ldots, x_n$ được định nghĩa như sau:

  $X_0 = 1;$

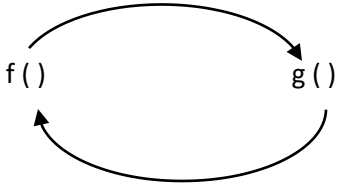
  $X_n = n^2X_0 + (n-1)^2X_1 + \ldots + 1^2X_{n-1} \quad , n \geq 1$

- Hàm đệ quy như sau:

```
long TinhXn(int   n)
{
        long  tam ;
        if (n == 0) return  1;
        tam  =  0;
        for (int  i =1; i <=n; i++)
                tam  +=  i*i*TinhXn (n – 1);
        return  tam;
}
```

Đệ quy hỗ tương

- Hàm đệ quy hỗ tương là hàm mà trong thân của hàm này có lời gọi hàm đến hàm kia và trong thân của hàm kia lại có lời gọi hàm tới hàm này.



- Thuật toán đệ quy hỗ tương như sau:

```
<kiểu dữ liệu hàm> <tên hàm 1> (<danh sách tham số>)
{
        // Thực hiện một số công việc (nếu có)
        <tên hàm 2> (<danh sách tham số>) ;
        // Thực hiện một số công việc (nếu có)
}
<kiểu dữ liệu hàm> <tên hàm 2> (<danh sách tham số>)
{
        // Thực hiện một số công việc (nếu có)
        <tên hàm 1> (<danh sách tham số>) ;
        // Thực hiện một số công việc (nếu có)
}
```

Đệ quy hỗ tương

- Ví dụ: Tính số hạng thứ n của hai dãy $\{X_n\}$, $\{Y_n\}$ được định nghĩa như sau:

  $X_0 = Y_0 = 1$;

  $X_n = X_{n-1} + Y_{n-1}$; $(n > 0)$

  $Y_n = n^2 X_{n-1} + Y_{n-1}$; $(n > 0)$

- Các hàm đệ quy như sau:

```
long TinhXn(int  n)
{
    if (n == 0 ) return 1;
    return  TinhXn(n – 1) + TinhYn(n – 1);
}

long TinhYn(int  n)
{
    if (n == 0 ) return 1;
    return  n*n*TinhXn(n – 1) + TinhYn(n – 1);
}
```

2. THUẬT TOÁN QUAY LUI

Backtracking Algorithm

NỘI DUNG

1.1. Sơ đồ thuật toán

1.2. Liệt kê các cấu hình tổ hợp cơ bản

- Liệt kê xâu nhị phân độ dài n
- Liệt kê tập con m phần tử của tập n phần tử
- Liệt kê hoán vị

1.3. Bài toán xếp hậu

SƠ ĐỒ THUẬT TOÁN

- Thuật toán quay lui (Backtracking Algorithm) là một thuật toán cơ bản được áp dụng để giải quyết nhiều vấn đề khác nhau.
- ***Bài toán liệt kê* (Q):** *Cho $A_1, A_2, \ldots, A_n$ là các tập hữu hạn. Ký hiệu*

  $$X = A_1 \times A_2 \times \ldots \times A_n = \{ (x_1, x_2, \ldots, x_n): \ x_i \in A_i \ , i=1, 2, \ldots, n\}.$$

  Giả sử P là tính chất cho trên X. Vấn đề đặt ra là liệt kê tất cả các phần tử của X thoả mãn tính chất P:

  $$D = \{ x = (x_1, x_2, \ldots, x_n) \in X: \ x \text{ thoả mãn tính chất } P \}.$$

- Các phần tử của tập *D* được gọi là các ***lời giải chấp nhận được.***

Ví dụ

- Tất cả các bài toán liệt kê tổ hợp cơ bản đều có thể phát biểu dưới dạng bài toán (Q)
- Bài toán liệt kê xâu nhị phân độ dài n dẫn về việc liệt kê các phần tử của tập

$$B^n = \{(x_1, \ldots, x_n): x_i \in \{0, 1\}, i=1, 2, \ldots, n\}.$$

- Bài toán liệt kê các tập con m phần tử của tập $N = \{1, 2, \ldots, n\}$ đòi hỏi liệt kê các phần tử của tập:

$$S(m,n) = \{(x_1,\ldots, x_m) \in N^m: 1 \leq x_1 < \ldots < x_m \leq n\}.$$

- Tập các hoán vị của các số tự nhiên 1, 2, ..., n là tập

$$\Pi_n = \{(x_1,\ldots, x_n) \in N^n: x_i \neq x_j ; i \neq j\}.$$

Lời giải bộ phận

- **Định nghĩa.** Ta gọi lời giải bộ phận cấp k ($0 \leq k \leq n$) là bộ có thứ tự gồm k thành phần

$$(a_1, a_2, \ldots, a_k),$$

trong đó $a_i \in A_i$, $i = 1, 2, \ldots, k$.

- Khi $k = 0$, lời giải bộ phận cấp 0 được ký hiệu là ( ) và còn được gọi là lời giải rỗng.
- Nếu $k = n$, ta có lời giải đầy đủ hay đơn giản là một lời giải của bài toán.

Ý tưởng chung

- Thuật toán quay lui được xây dựng dựa trên việc xây dựng dần từng thành phần của lời giải.
- Thuật toán bắt đầu từ lời giải rỗng (). Trên cơ sở tính chất P ta xác định được những phần tử nào của tập A_1 có thể chọn vào vị trí thứ nhất của lời giải. Những phần tử như vậy ta sẽ gọi là những ứng cử viên (viết tắt là UCV) vào vị trí thứ nhất của lời giải.
- Ký hiệu tập các UCV vào vị trí thứ nhất của lời giải là S_1. Lấy $a_1 \in S_1$, bổ sung nó vào lời giải rỗng đang có ta thu được lời giải bộ phận cấp 1: (a_1).

Bước tổng quát

- Tại bước tổng quát, giả sử ta đang có lời giải bộ phận cấp k-1: $(a_1, a_2, ..., a_{k-1})$.
- Trên cơ sở tính chất P ta xác định được những phần tử nào của tập A_k có thể chọn vào vị trí thứ k của lời giải.
- Những phần tử như vậy ta sẽ gọi là những ứng cử viên (viết tắt là UCV) vào vị trí thứ k của lời giải khi k-1 thành phần đầu của nó đã được chọn là $(a_1, a_2,..., a_{k-1})$. Ký hiệu tập các ứng cử viên này là S_k.

Xét hai tình huống

- **Tình huống 1:** $S_k \neq \varnothing$. Khi đó lấy $a_k \in S_k$, bổ sung nó vào lời giải bộ phận cấp k-1 đang có

$$(a_1, a_2, ..., a_{k-1})$$

ta thu được lời giải bộ phận cấp k:

$$(a_1, a_2, ..., a_{k-1}, a_k).$$

- Khi đó
  - Nếu $k = n$ thì ta thu được một lời giải,
  - Nếu $k < n$, ta tiếp tục đi xây dựng thành phần thứ k+1 của lời giải.

Tình huống ngõ cụt

- **Tình huống 2:** $S_k = \varnothing$. Điều đó có nghĩa là lời giải bộ phận (a_1, a_2, ..., a_{k-1}) không thể tiếp tục phát triển thành lời giải đầy đủ. Trong tình huống này ta quay trở lại tìm ứng cử viên mới vào vị trí thứ k-1 của lời giải.
- Nếu tìm thấy UCV như vậy, thì bổ sung nó vào vị trí thứ k-1 rồi lại tiếp tục đi xây dựng thành phần thứ k.
- Nếu không tìm được thì ta lại quay trở lại thêm một bước nữa tìm UCV mới vào vị trí thứ k-2, ... Nếu quay lại tận lời giải rỗng mà vẫn không tìm được UCV mới vào vị trí thứ 1, thì thuật toán kết thúc.

Thuật toán quay lui

```
procedure Bactrack(k: integer);
begin
        Xây dựng S_k;
        for y ∈ S_k do (* Với mỗi UCV y từ S_k *)
        begin
            a_k := y;
            if k = n then <Ghi nhận lời giải (a_1, a_2, ..., a_k) >
            else Backtrack(k+1);
        end;
end;
```

Lệnh gọi để thực hiện thuật toán quay lui là: Bactrack(1)

Hai vấn đề mấu chốt

- Để cài đặt thuật toán quay lui giải các bài toán tổ hợp cụ thể ta cần giải quyết hai vấn đề cơ bản sau:
  - Tìm thuật toán xây dựng các tập UCV S_k.
  - Tìm cách mô tả các tập này để có thể cài đặt thao tác liệt kê các phần tử của chúng (cài đặt vòng lặp qui ước **for y $\in$ S_k do).**
- Hiệu quả của thuật toán liệt kê phụ thuộc vào việc ta có xác định được chính xác các tập UCV này hay không.

Chú ý

- Nếu chỉ cần tìm một lời giải thì cần tìm cách chấm dứt các thủ tục gọi đệ qui lồng nhau sinh bởi lệnh gọi Backtrack(1) sau khi ghi nhận được lời giải đầu tiên.
- Nếu kết thúc thuật toán mà ta không thu được một lời giải nào thì điều đó có nghĩa là bài toán không có lời giải.

Chú ý

- Thuật toán dễ dàng mở rộng cho bài toán liệt kê trong đó lời giải có thể mô tả như là bộ (a_1, a_2, ..., a_n,...) độ dài hữu hạn, tuy nhiên giá trị của độ dài là không biết trước và các lời giải cũng không nhất thiết phải có cùng độ dài.
- Khi đó chỉ cần sửa lại câu lệnh

  if k = n then <Ghi nhận lời giải (a_1, a_2, ..., a_k) >

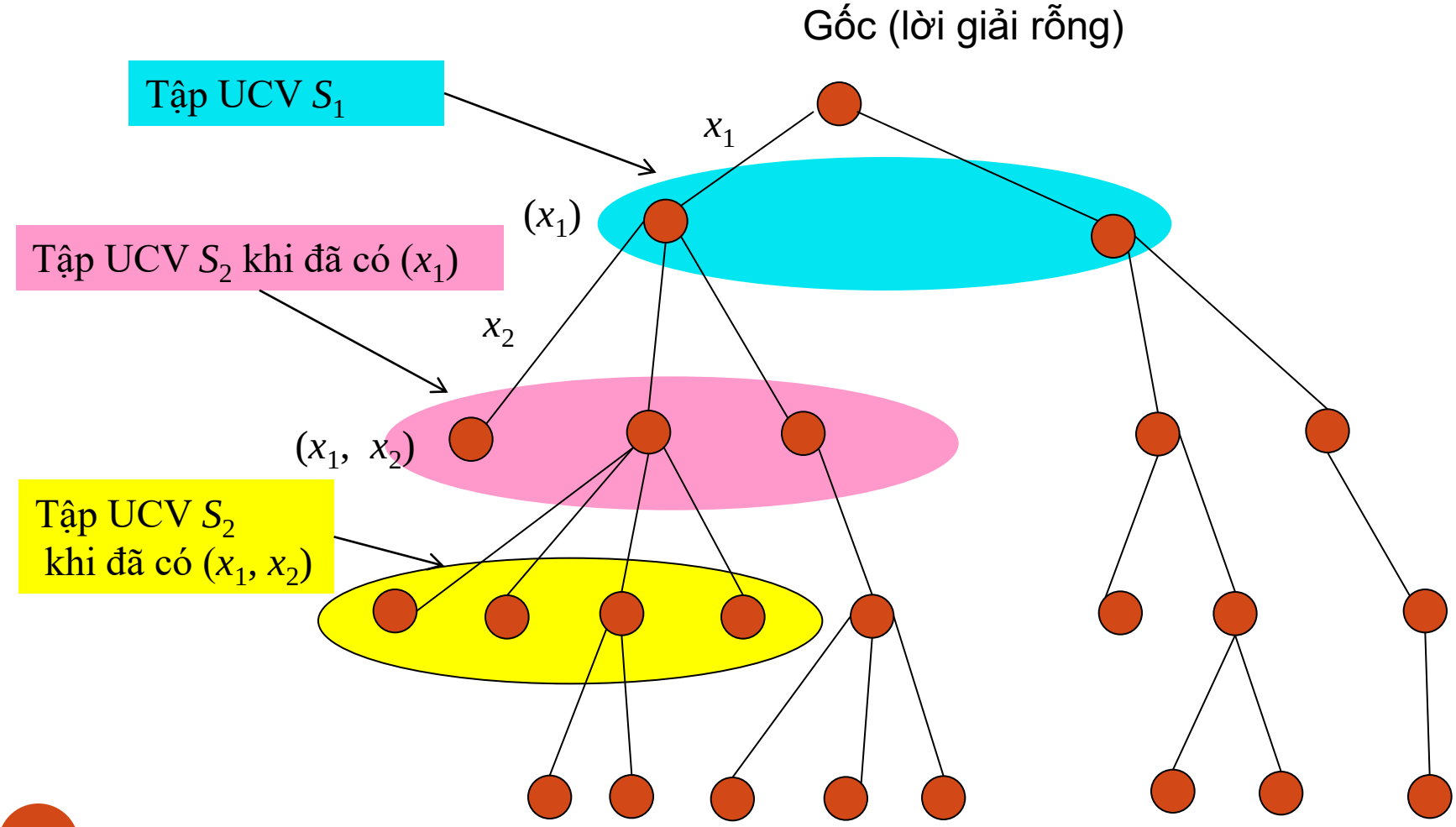
  else Backtrack(k+1);

  thành

  if <(a_1, a_2, ..., a_k) là lời giải> then <Ghi nhận (a_1, a_2, ..., a_k) >

  else Backtrack(k+1);
- Cần xây dựng hàm nhận biết (a_1, a_2, ..., a_k) đã là lời giải hay chưa.

Cây liệt kê lời giải theo thuật toán quay lui



Liệt kê xâu nhị phân độ dài n

Liệt kê xâu nhị phân độ dài n

- Bài toán liệt kê xâu nhị phân độ dài n dẫn về việc liệt kê các phần tử của tập

$$B^n = \{(x_1, ..., x_n): x_i \in \{0, 1\}, i=1, 2, ..., n\}.$$

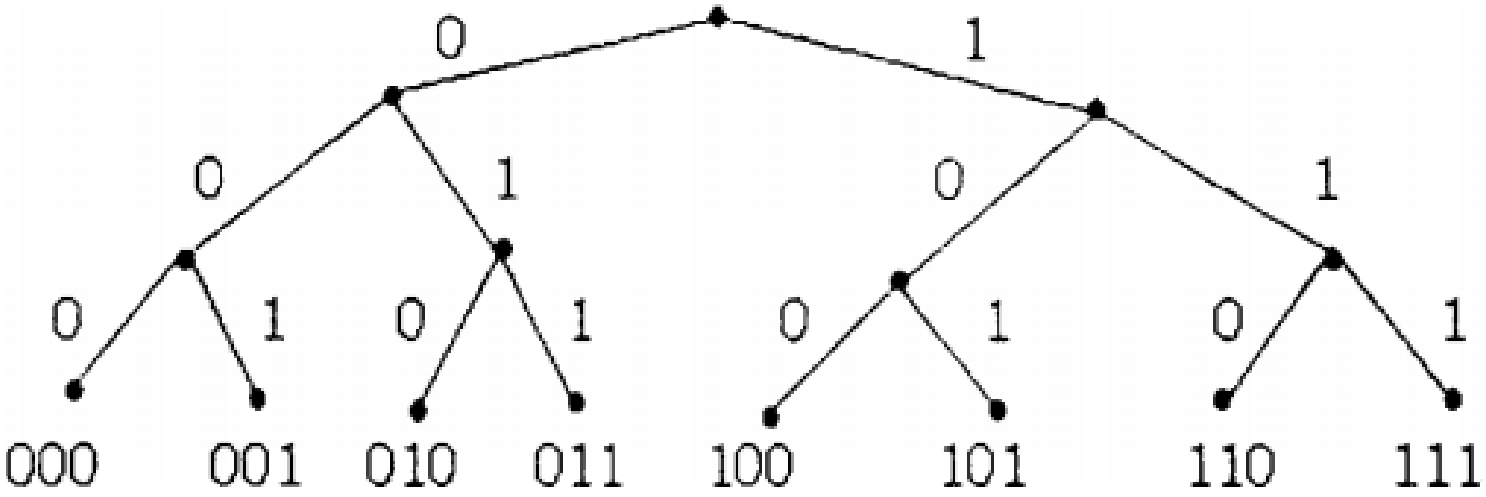
- Ta xét cách giải quyết hai vấn đề cơ bản để cài đặt thuật toán quay lui:
  - Rõ ràng ta có $S_1 = \{0, 1\}$. Giả sử đã có xâu nhị phân cấp k-1 (b_1, ..., b_{k-1}), khi đó rõ ràng $S_k = \{0,1\}$. Như vậy, tập các UCV vào các vị trí của lời giải được đã xác định.
  - Để cài đặt vòng lặp liệt kê các phần tử của S_k, dễ thấy là ta có thể sử dụng vòng lặp for

trên PASCAL: **for y:= 0 to 1 do**

hoặc trên C: **for (y=0;y<=1;y++)**

Liệt kê xâu nhị phân độ dài n

- Cây liệt kê dãy nhị phân độ dài 3



Chương trình trên C

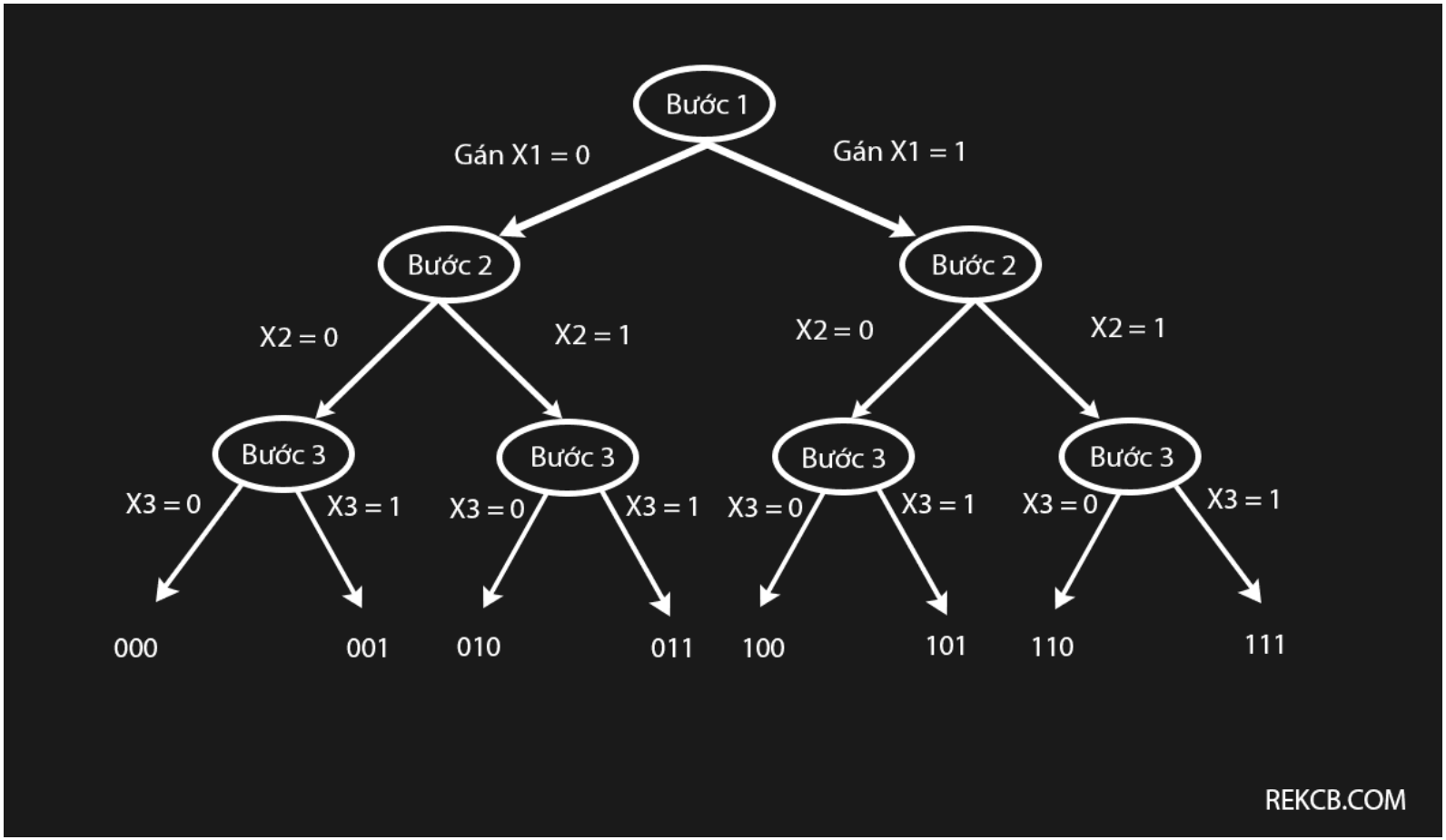
```
#include <stdio.h>
#include <conio.h>
using namespace std;
int  n, count;
int b[20];

void Ghinhan()
{
    count++;
    printf("Xau thu %2d. ",count);
    for(int i=1 ; i<= n ;i++)
        printf("%d  ", b[i]);
    printf("\n");
}
```

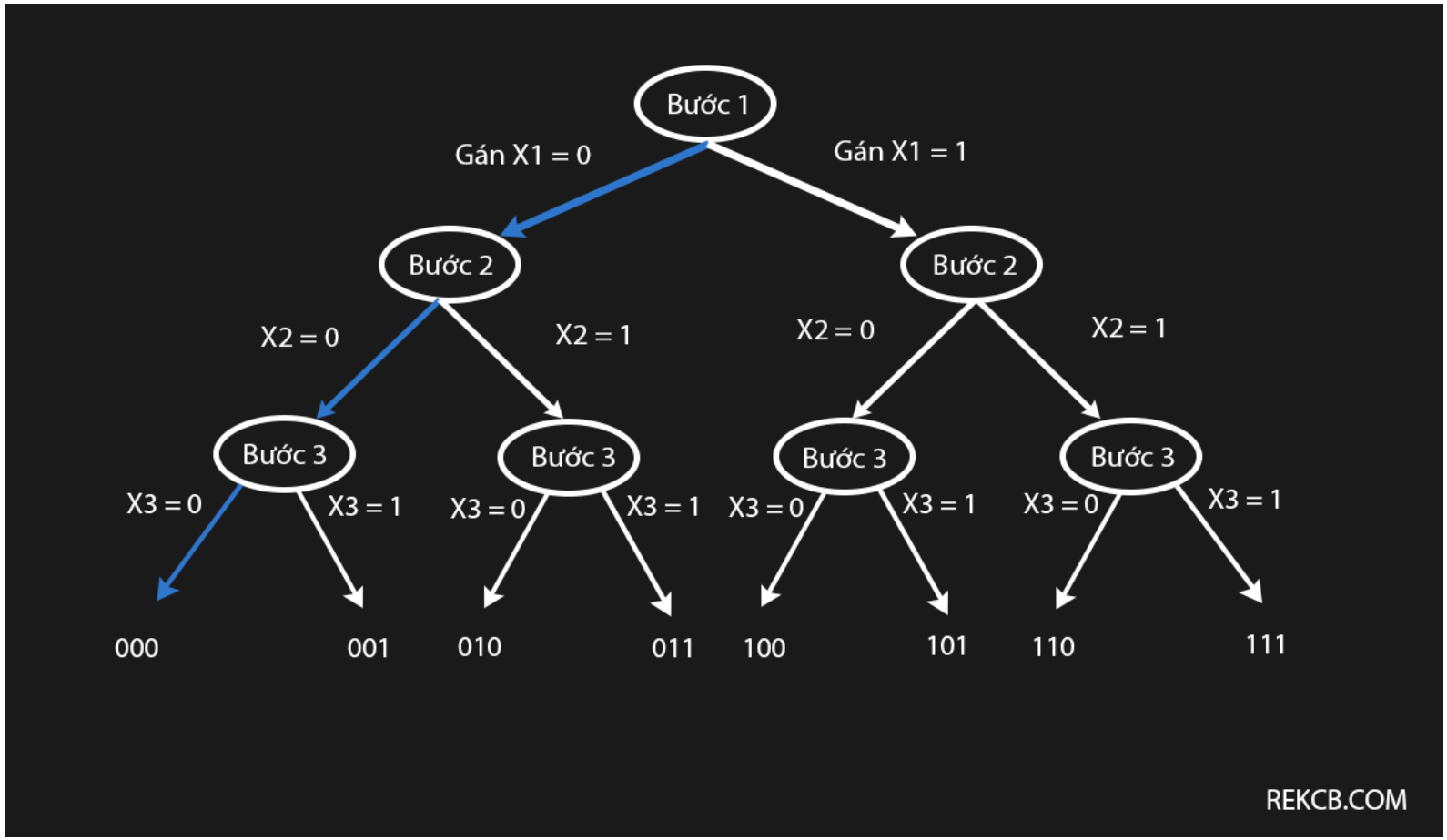
```
void  Xau(int i)
{
    for (int j = 0; j<=1; j++)
     { b[i] = j;
        if (i==n)  Ghinhan();
        else  Xau(i+1); }
}

int main()
{
     printf("n = ");
     scanf("%d",&n);
     count = 0; Xau(1);
     printf("Count = %d ", count);
}
```

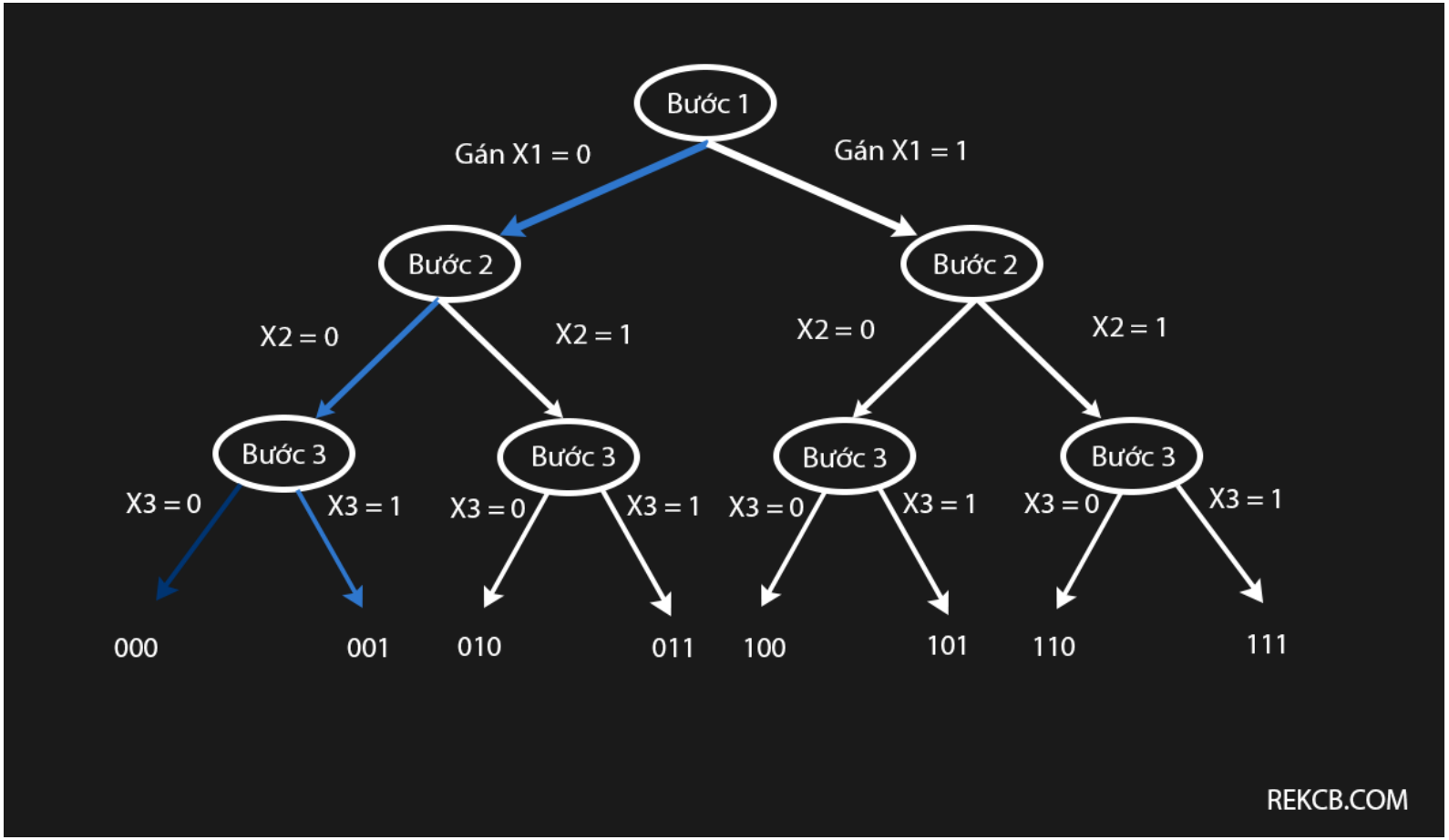
Cây liệt kê dãy nhị phân độ dài 3



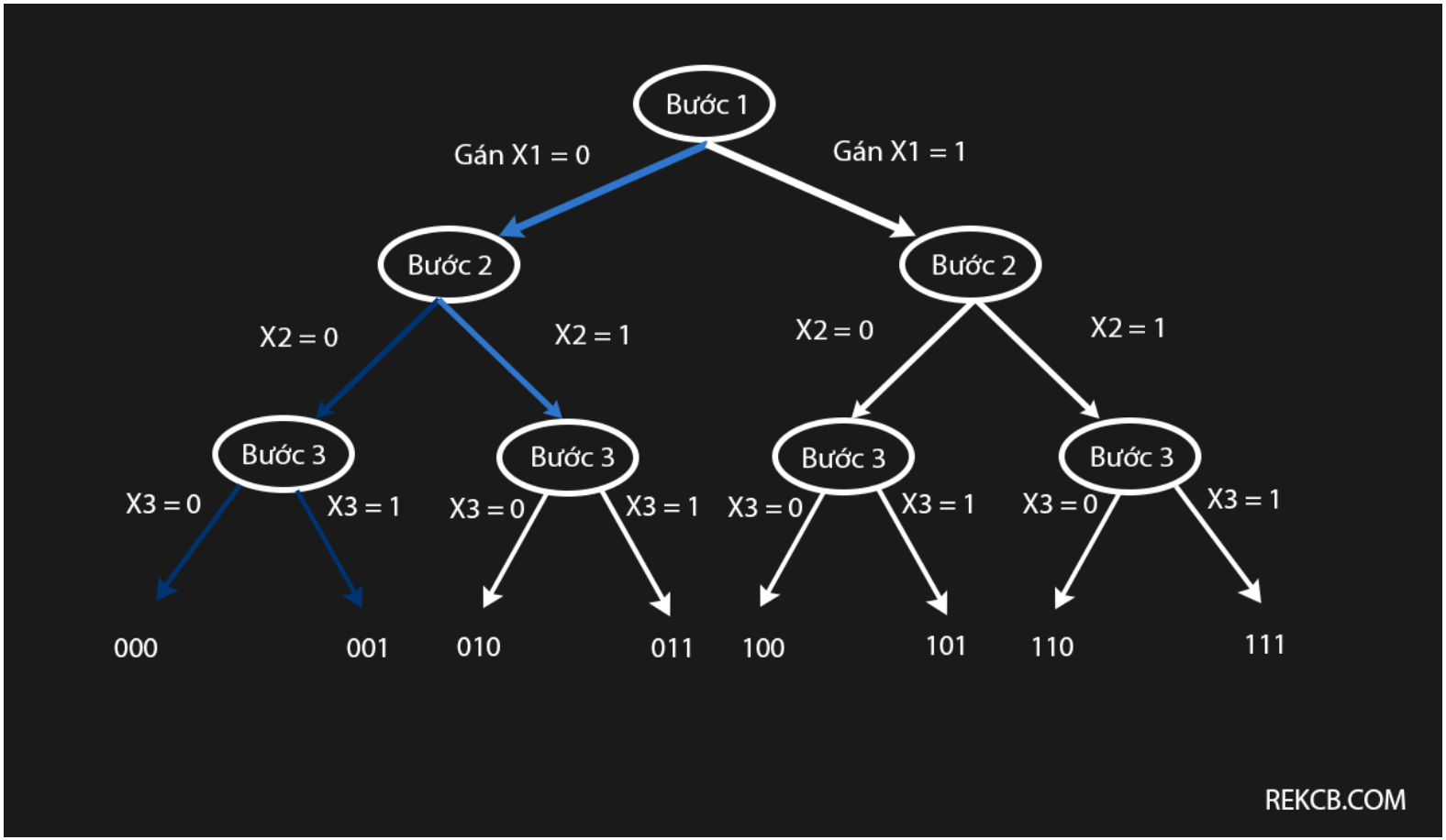
Cây liệt kê dãy nhị phân độ dài 3



Cây liệt kê dãy nhị phân độ dài 3



Cây liệt kê dãy nhị phân độ dài 3



Liệt kê các *m*-tập con của *n*-tập

Liệt kê các m-tập con của n-tập

- **Bài toán:** Liệt kê các tập con m phần tử của tập $N = \{1, 2, ..., n\}$.

- Bài toán dẫn về: Liệt kê các phần tử của tập:

$$S(m,n)=\{(x_1,..., x_m)\in N^m: 1 \le x_1<...<x_m\le n\}$$

Giải quyết 2 vấn đề mấu chốt

- Từ điều kiện: $1 \le a_1 < a_2 < \ldots < a_m \le n$

  suy ra $S_1 = \{1, 2, \ldots, n\text{-}(m\text{-}1)\}$.
- Giả sử đã có tập con $(a_1, \ldots, a_{k\text{-}1})$.

  Từ điều kiện $a_{k\text{-}1} < a_k < \ldots < a_m \le n$, ta suy ra

  $S_k = \{a_{k\text{-}1}+1, a_{k\text{-}1}+2, \ldots, n\text{-}(m\text{-}k)\}$.
- Để cài đặt vòng lặp liệt kê các phần tử của S_k, dễ thấy là ta có thể sử dụng vòng lặp

  của PASCAL: **for** y:= a[k-1]+1 **to** n-m+k **do ...**

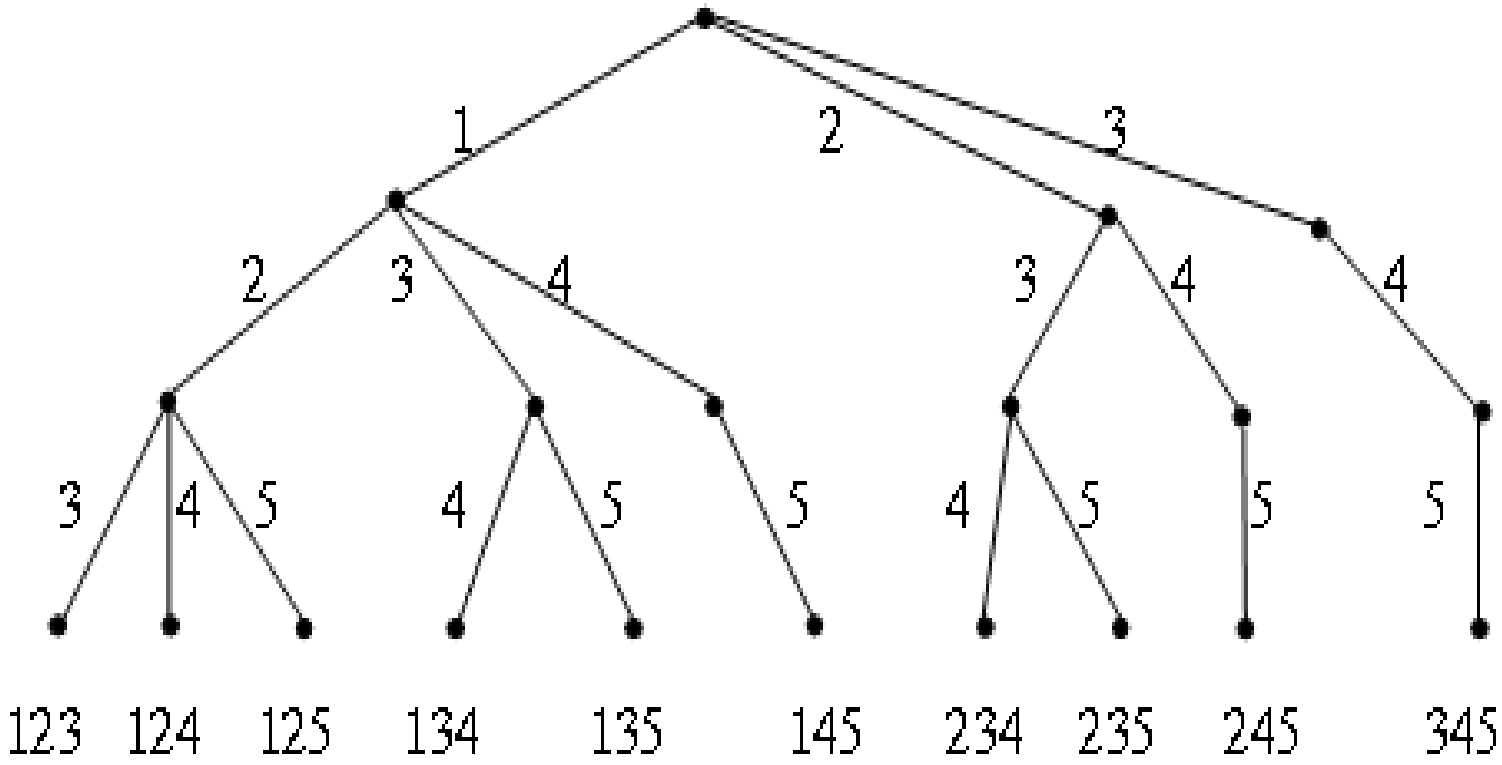
  của C: **for (**y=a[k-1]+1;y<=n-m+k;y++**) ...**

Chương trình trên C

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
int  n, m, count;
int a[20];
void Ghinhan() {
   int i, j;
    count++;
    printf("Tap con thu %i. ",count);
    for  (i=1 ; i<= m ;i++)
        printf("%i  ", a[i]);
    printf("\n");
}
```

```
void  MSet(int i){
   int j;
   for (j = a[i-1] +1; j<= n-m+i; j++) {
        a[i] = j;
        if (i==m)  Ghinhan();
        else MSet(i+1);
    }
}
int main() {
     printf("n, m = ");
     scanf("%i %i",&n, &m); printf("\n");
     a[0]=0; count = 0; MSet(1);
     printf("Count = %d ", count);
     getch();
}
```

Cây liệt kê S(5,3)



Liệt kê hoán vị

Liệt kê hoán vị

- Tập các hoán vị của các số tự nhiên 1, 2, ..., n là tập

$$\Pi_n = \{(x_1,\ldots, x_n) \in N^n:\ x_i \neq x_j\ ,\ i \neq j\ \}.$$

- **Bài toán: Liệt kê tất cả các phần tử của Π_n**

Giải quyết 2 vấn đề mấu chốt

- Rõ ràng $S_1 = N$. Giả sử ta đang có hoán vị bộ phận (a_1, a_2, ..., a_{k-1}), từ điều kiện $a_i \neq a_j$, với mọi $i \neq j$ ta suy ra

$$S_k = N \setminus \{ a_1, a_2, ..., a_{k-1} \}.$$

- Như vậy ta đã có cách xác định được tập các UCV vào các vị trí của lời giải.
- Mô tả S_k?

Mô tả S_k

Xây dựng hàm nhận biết UCV:

```
function UCV(j,k: integer): boolean;
(* UCV nhận giá trị true khi và chỉ khi j ∈ Sk *)
var i: integer;
begin
   for i:=1 to  k-1 do
       if j = a[i] then
       begin
            UCV:= false; exit;
       end;
   UCV:= true;
end;
```

Mô tả S_k

- Câu lệnh qui ước "**for y ∈ S_k do** " có thể cài đặt như sau

```
for y:=1 to n do
        if UCV(y,k) then
        begin
                (*  y là UCV vào vị trí k  *)
                . . .
        end
```

Chương trình trên C

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
int  n, count;
int b[20];

int Ghinhan() {
   int i, j;
    count++;
    printf("Hoan vi thu %i. ",count);
    for  (i=1 ; i<= n ;i++) {
          printf("%i  ", b[i]);
    }
    printf("\n");
}
```

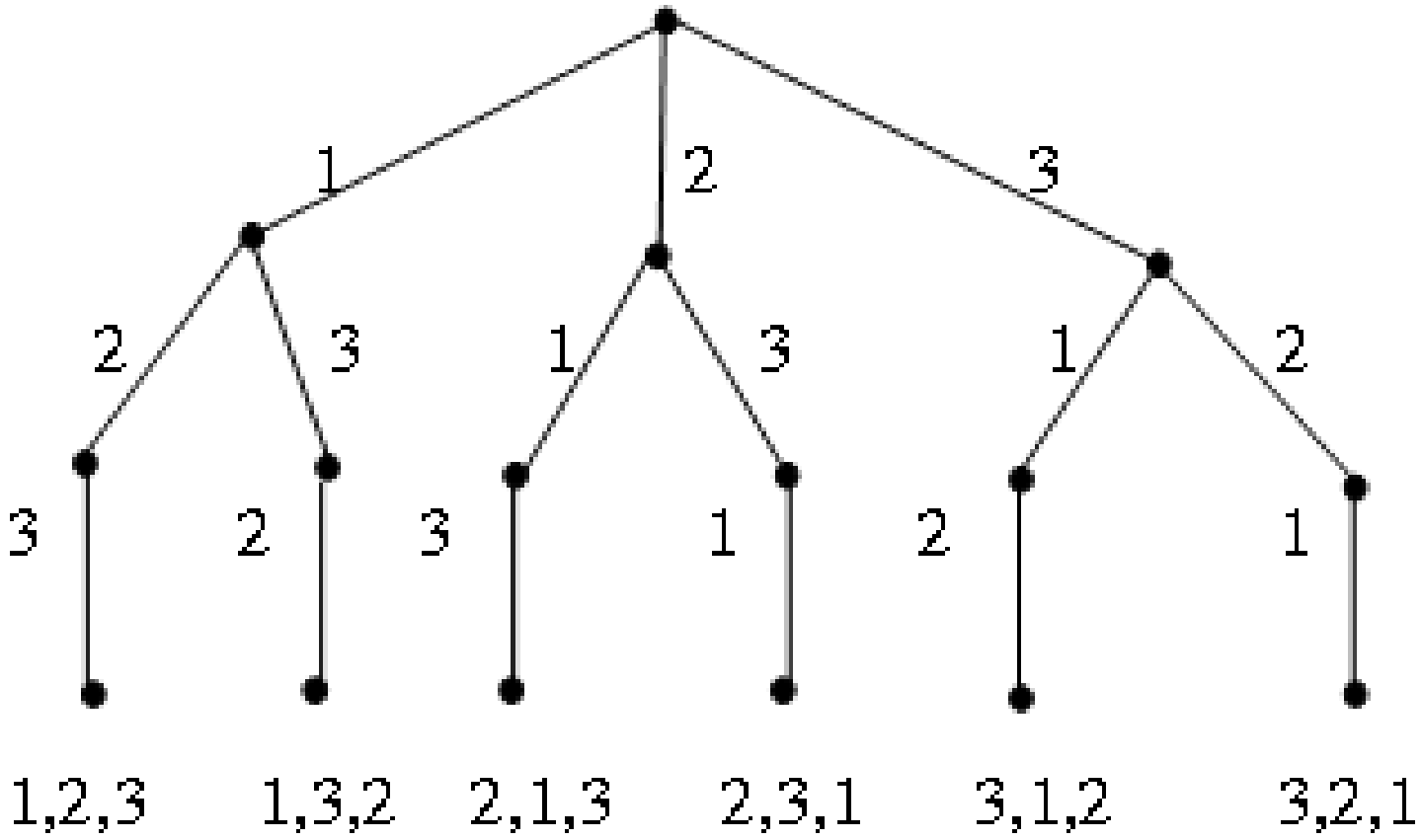
```
int UCV(int j, int k)
{
    int i;
    for (i=1; i<=k-1; i++)
          if (j == b[i]) return 0;
    return 1;
}
```

Chương trình trên C

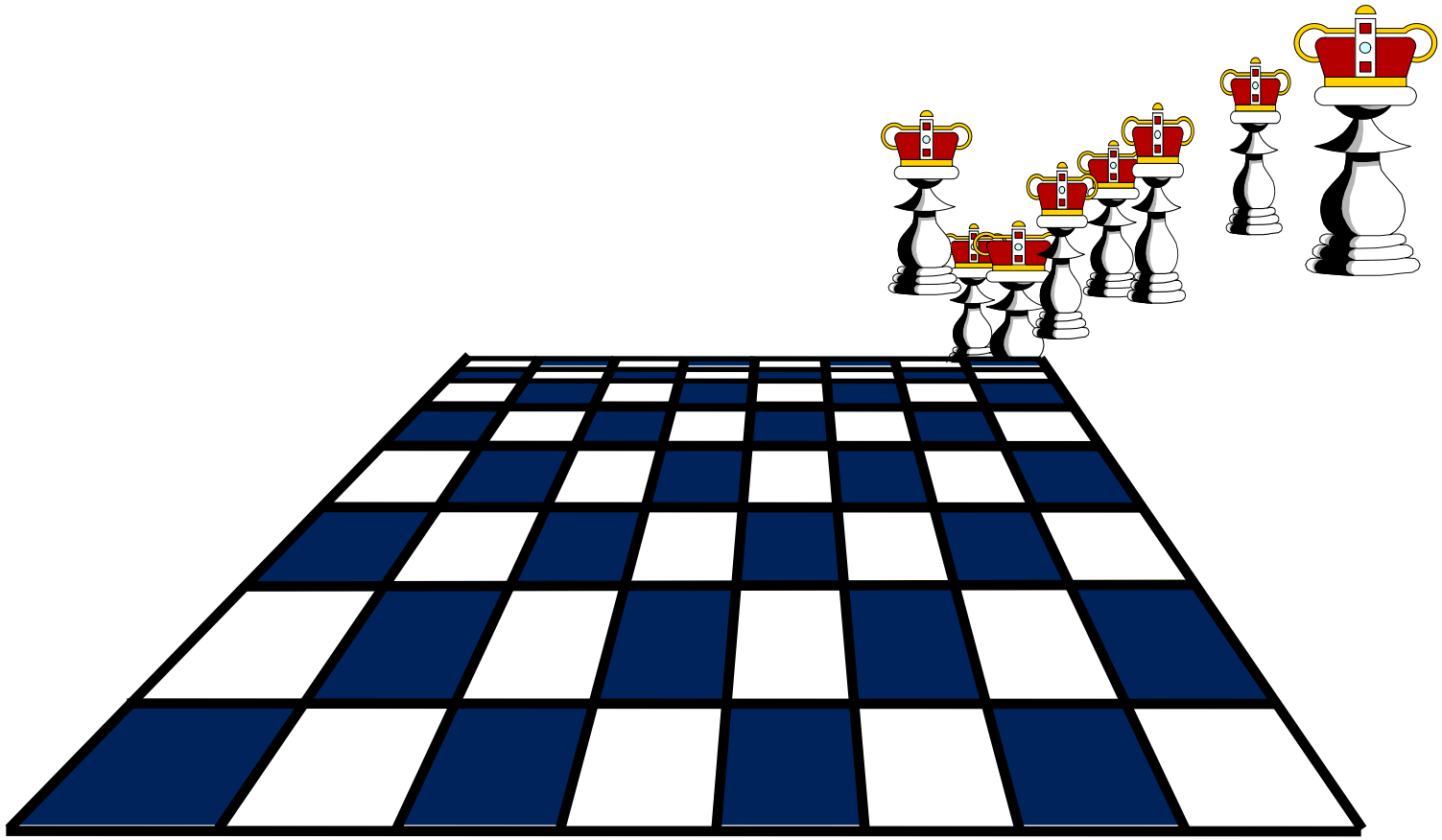
```
int  Hoanvi(int i){
   int j;
   for (j = 1; j<=n; j++)
      if (UCV(j,i)==1)
      {  b[i] = j;
         if (i==n)  Ghinhan();
         else  Hoanvi(i+1);
    }
}
```

```
int main() {
    printf("==========\n");
    printf("n = ");
    scanf("%i",&n);
     printf("\n");
     count = 0; Hoanvi(1);
     printf("Count = %d ", count);
     getch();
}
```

Cây liệt kê hoán vị của 1, 2, 3

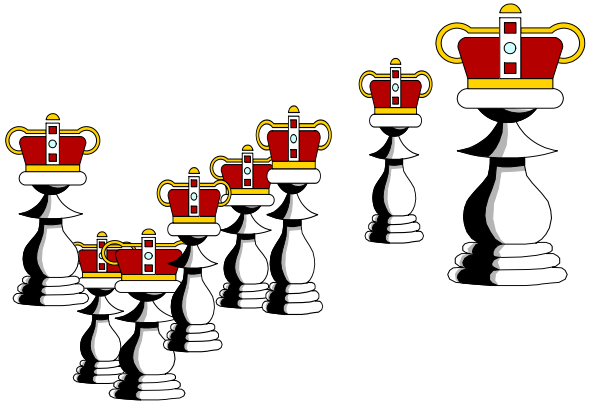


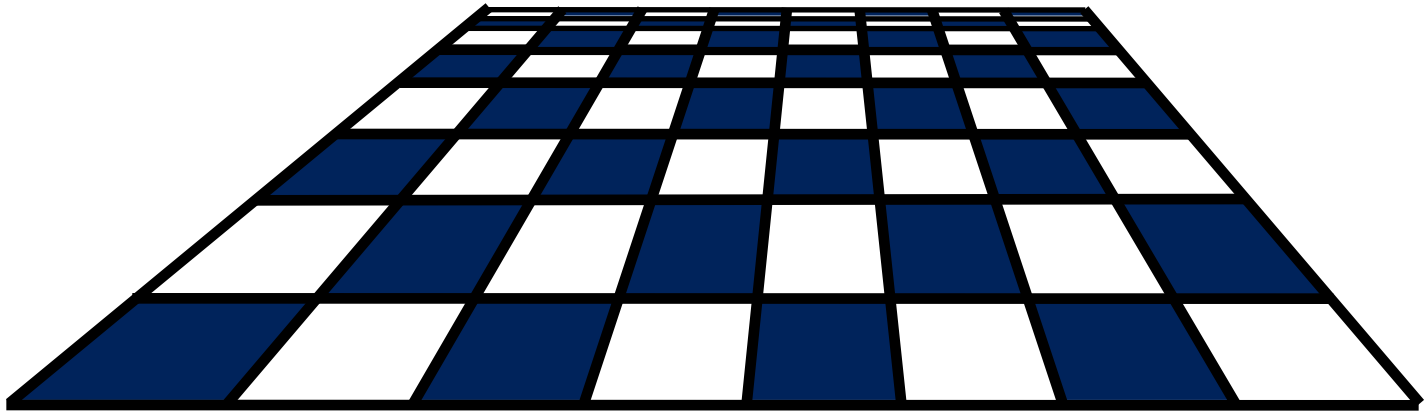
The *n*-Queens Problem



The *n*-Queens Problem

- Giả sử ta có 8 con hậu…
- …và bàn cờ quốc tế

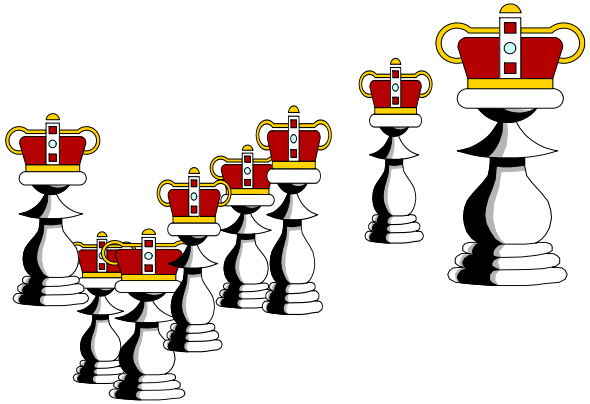


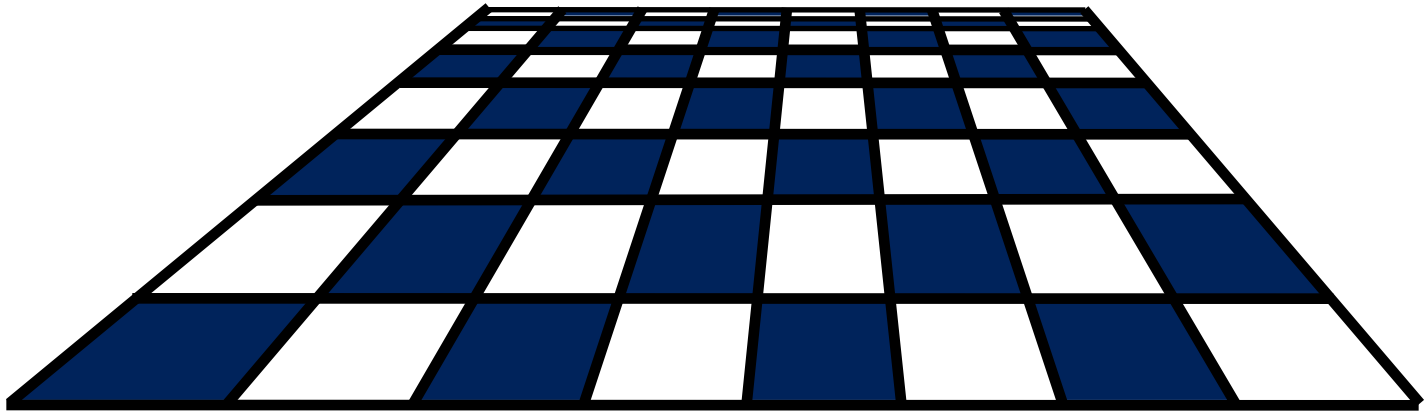


The *n*-Queens Problem

Có thể xếp các con hậu sao cho không có hai con nào ăn nhau hay không

?

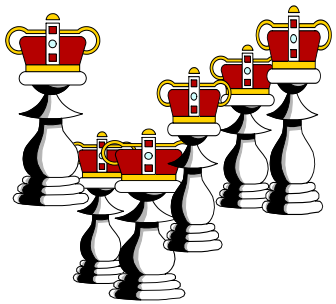


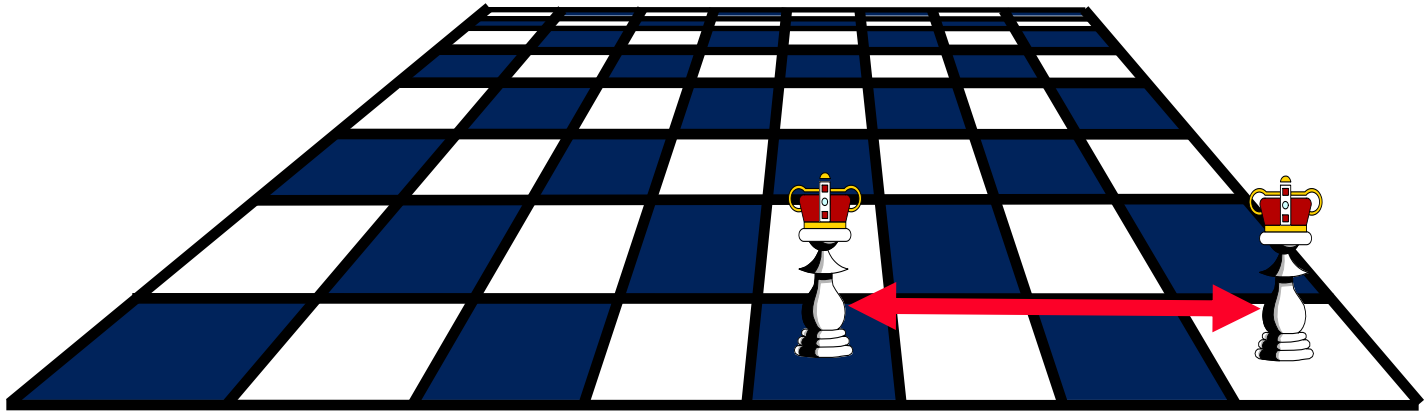


The n-Queens Problem

Hai con hậu bất kỳ không được xếp trên cùng một dòng

…

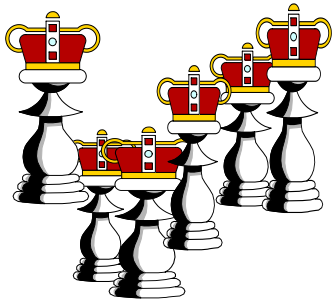


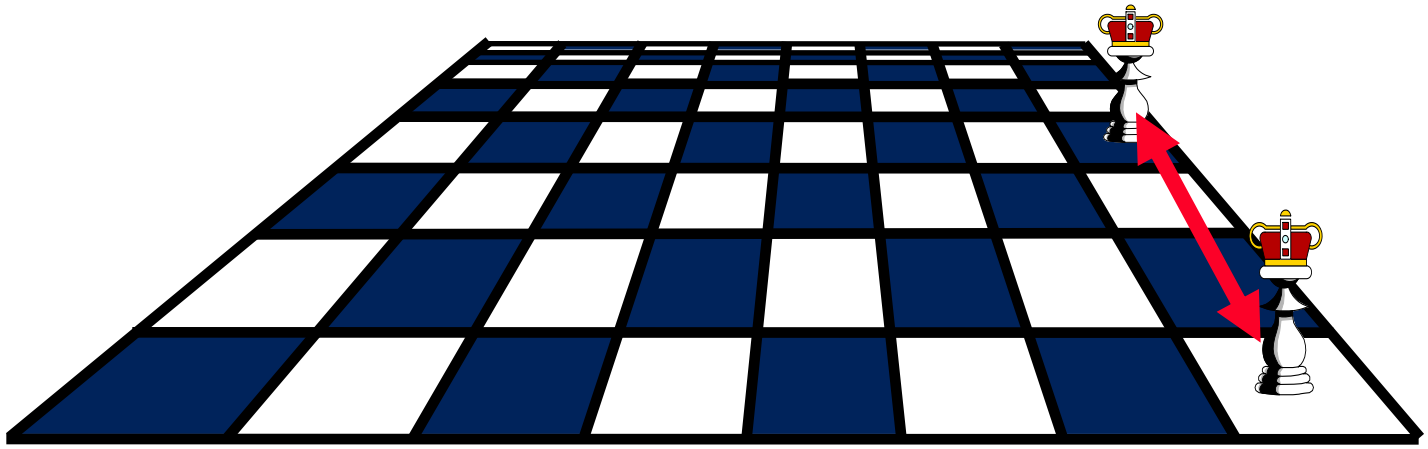


The n-Queens Problem

Hai con hậu bất kỳ không được xếp trên cùng một cột

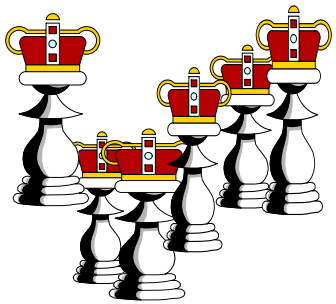
…

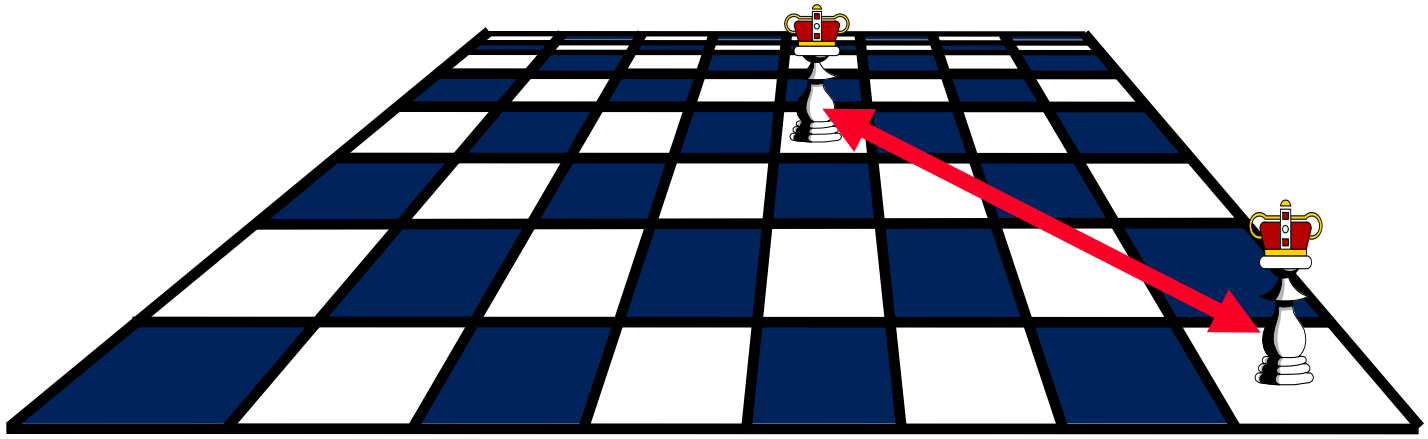




The n-Queens Problem

Hai con hậu bất kỳ không được xếp trên cùng một đường chéo!

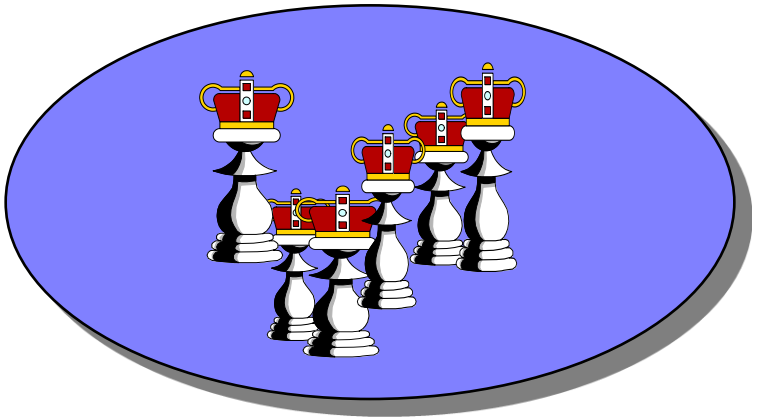




The n-Queens Problem

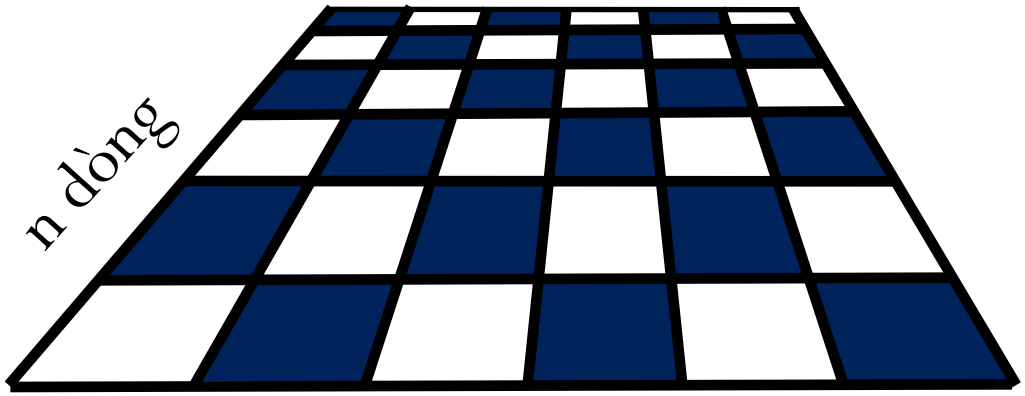
Kích thước n*n

n con hậu



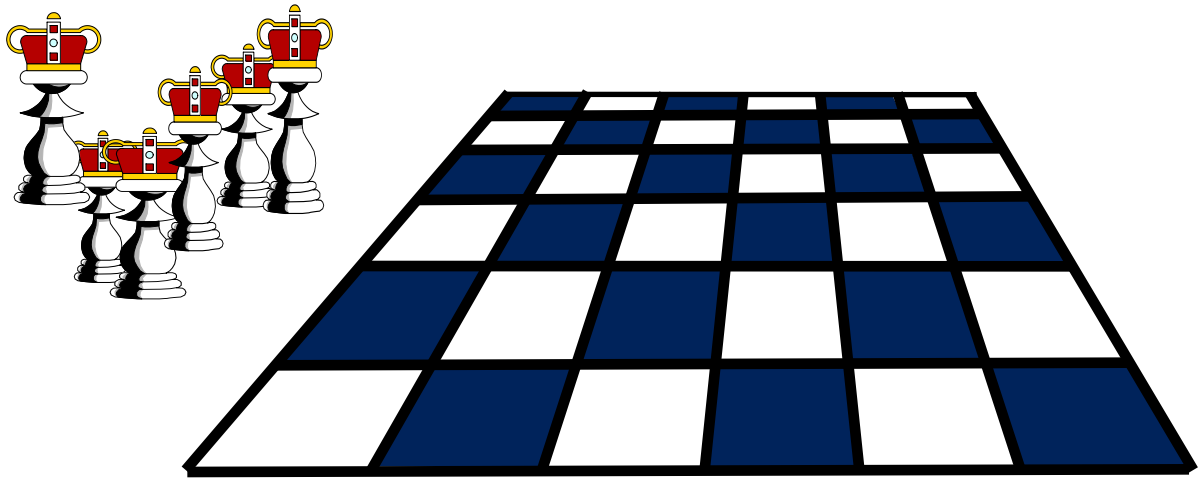
n cột

n dòng



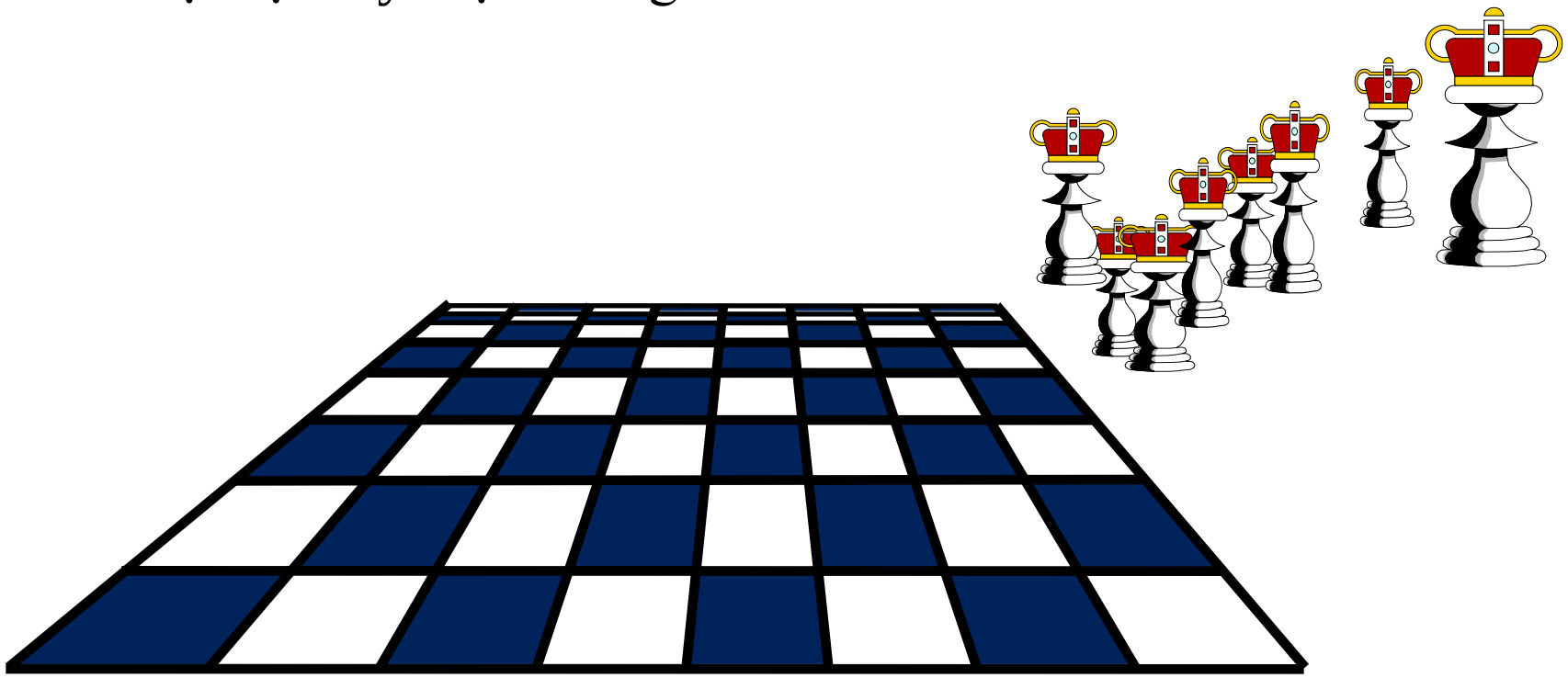
The n-Queens Problem

Xét bài toán xếp n con hậu
lên bàn cờ kích thước n x n.



Bài toán xếp hậu

- Liệt kê tất cả các cách xếp *n* quân Hậu trên bàn cờ *n*×*n* sao cho chúng không ăn được lẫn nhau, nghĩa là sao cho không có hai con nào trong số chúng nằm trên cùng một dòng hay một cột hay một đường chéo của bàn cờ.



Biểu diễn lời giải

- Đánh số các cột và dòng của bàn cờ từ 1 đến n. Một cách xếp hậu có thể biểu diễn bởi bộ có n thành phần $(a_1, a_2, ..., a_n)$, trong đó a_i là toạ độ cột của con Hậu ở dòng i.
- Các điều kiện đặt ra đối với bộ $(a_1, a_2, ..., a_n)$:
  - **$a_i \neq a_j$, với mọi $i \neq j$ (nghĩa là hai con hậu ở hai dòng i và j không được nằm trên cùng một cột);**
  - **$|a_i - a_j| \neq |i - j|$, với mọi $i \neq j$ (nghĩa là hai con hậu ở hai ô (a_i, i) và (a_j, j) không được nằm trên cùng một đường chéo).**

Phát biểu bài toán

- Như vậy bài toán xếp Hậu dẫn về bài toán liệt kê các phần tử của tập:

$$D=\{(a_1, a_2, ..., a_n)\in N^n:$$

$$a_i \neq a_j \text{ và } |a_i - a_j| \neq |i - j|, i \neq j\}.$$

Hàm nhận biết ứng cử viên

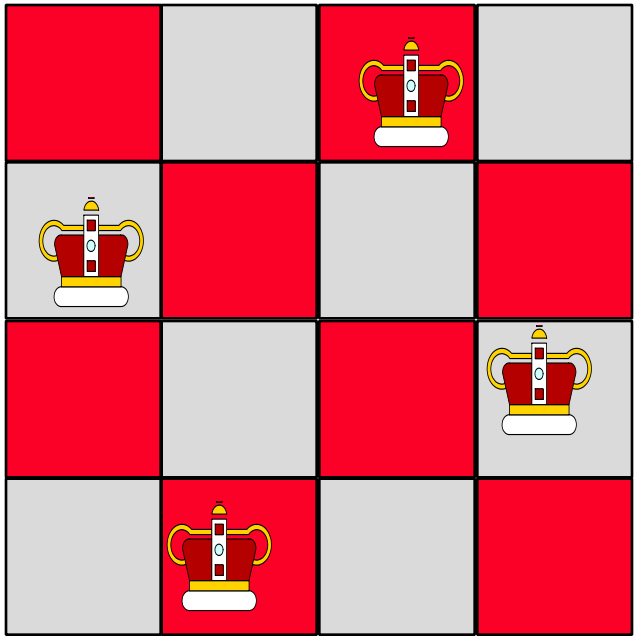
```
int UCVh(int j, int k) {
// UCVh nhận giá trị 1
// khi và chỉ khi j ∈ Sk
   int i;
   for (i=1; i<k; i++)
     if ((j == a[i])||(fabs(j-a[i])== k-i)) return 0;
   return 1;
}
```

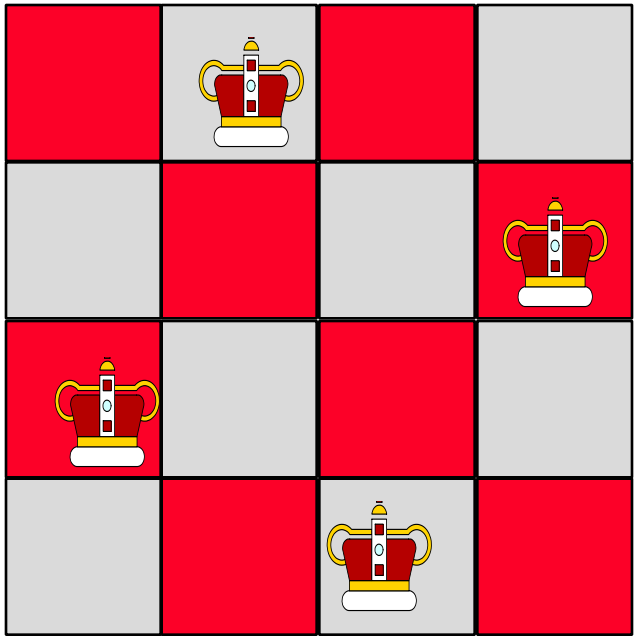
Chương trình trên C

```
#include <stdio.h>
#include <math.h>
int   n, count;
int   a[20];
int Ghinhan() {
   int i;
   count++; printf("%i. ",count);
   for (i=1; i<=n;i++) printf("%i  ",a[i]);
   printf("\n");
}

int UCVh(int j, int k) {
/* UCVh nhận giá trị 1 khi và chỉ khi j ∈ Sk */
   int i;
   for (i=1; i<k; i++)
       if ((j==a[i])||(fabs(j-a[i])==k-i)) return 0;
   return 1;
}
```

```
int Hau(int i){   int j;
   for (j=1; j<=n; j++)
         if  (UCVh(j, i)){
             a[i] = j;
             if  (i == n) Ghinhan();
             else Hau(i+1);
         }
}
int main() {
    printf("Input n = "); scanf("%i",&n);
    printf("\n==== RESULT ====\n");
    count = 0; Hau(1);
    if (count == 0) printf("No solution!\n");
    getch();
    printf("\n Press Enter to finish... ");
    getchar();
}
```

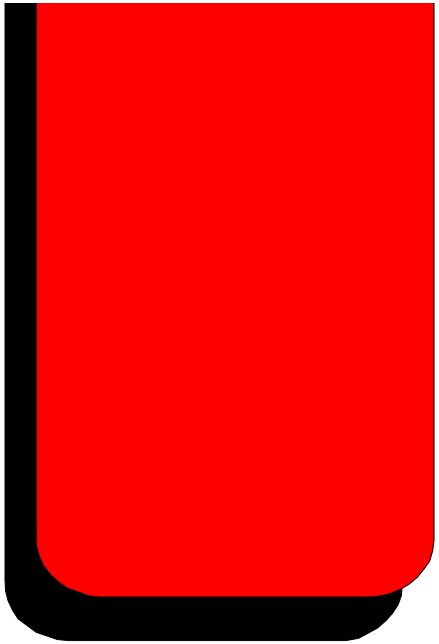


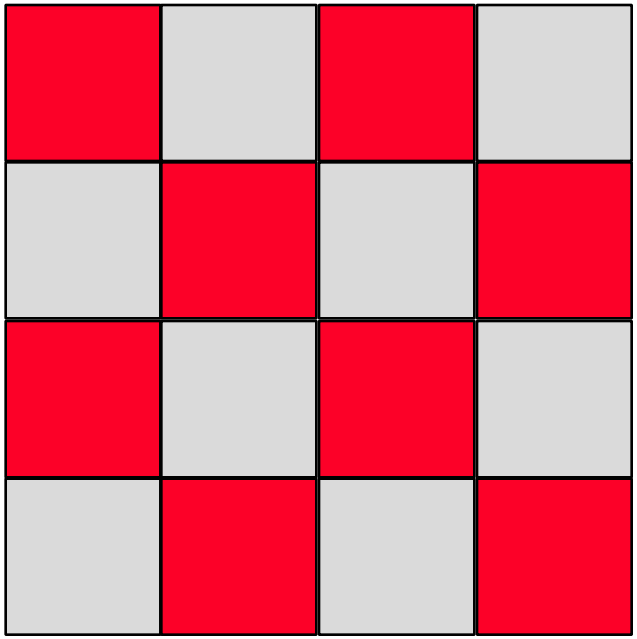


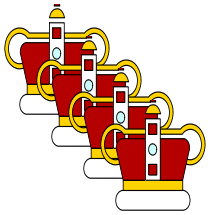
Chú ý

- Rõ ràng là bài toán xếp hậu không phải là luôn có lời giải, chẳng hạn bài toán không có lời giải khi n=2, 3. Do đó điều này cần được thông báo khi kết thúc thuật toán.
- Thuật toán trình bày ở trên là chưa hiệu quả. Nguyên nhân là ta đã không xác định được chính xác các tập UCV vào các vị trí của lời giải.

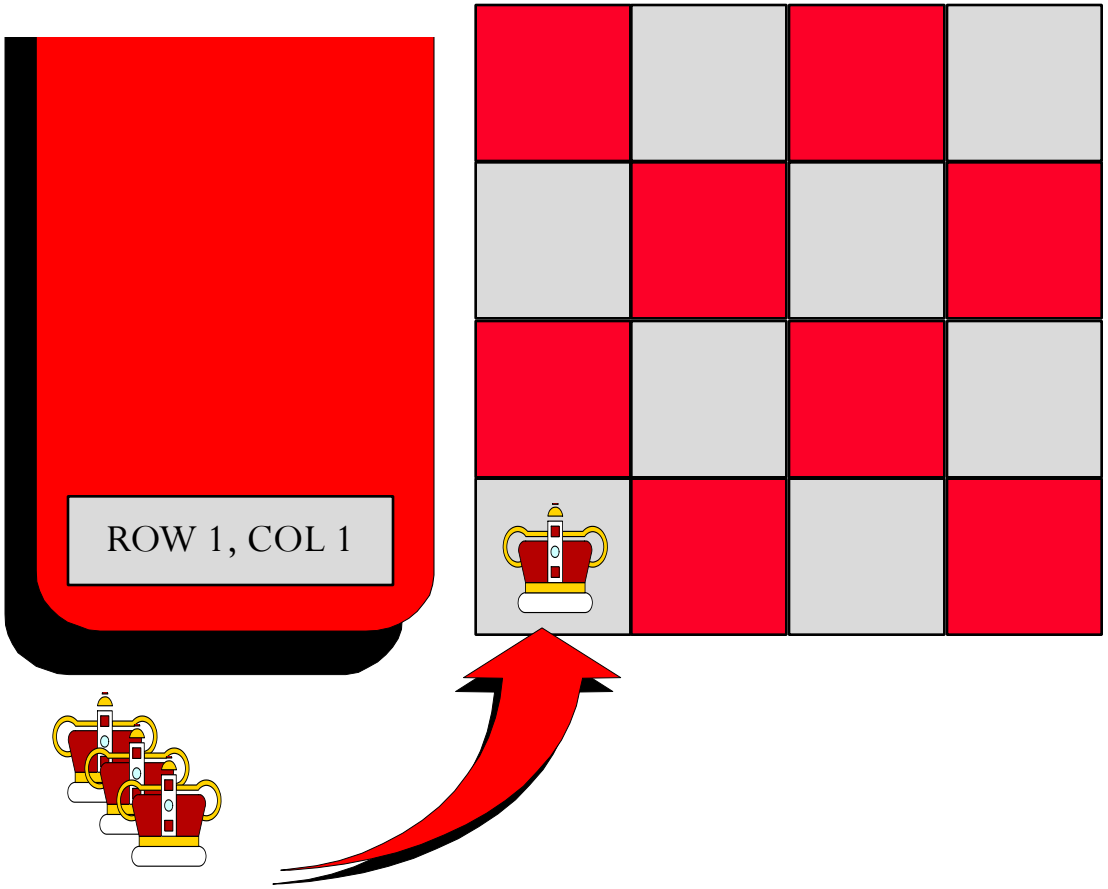
Thuật toán làm việc như thế nào





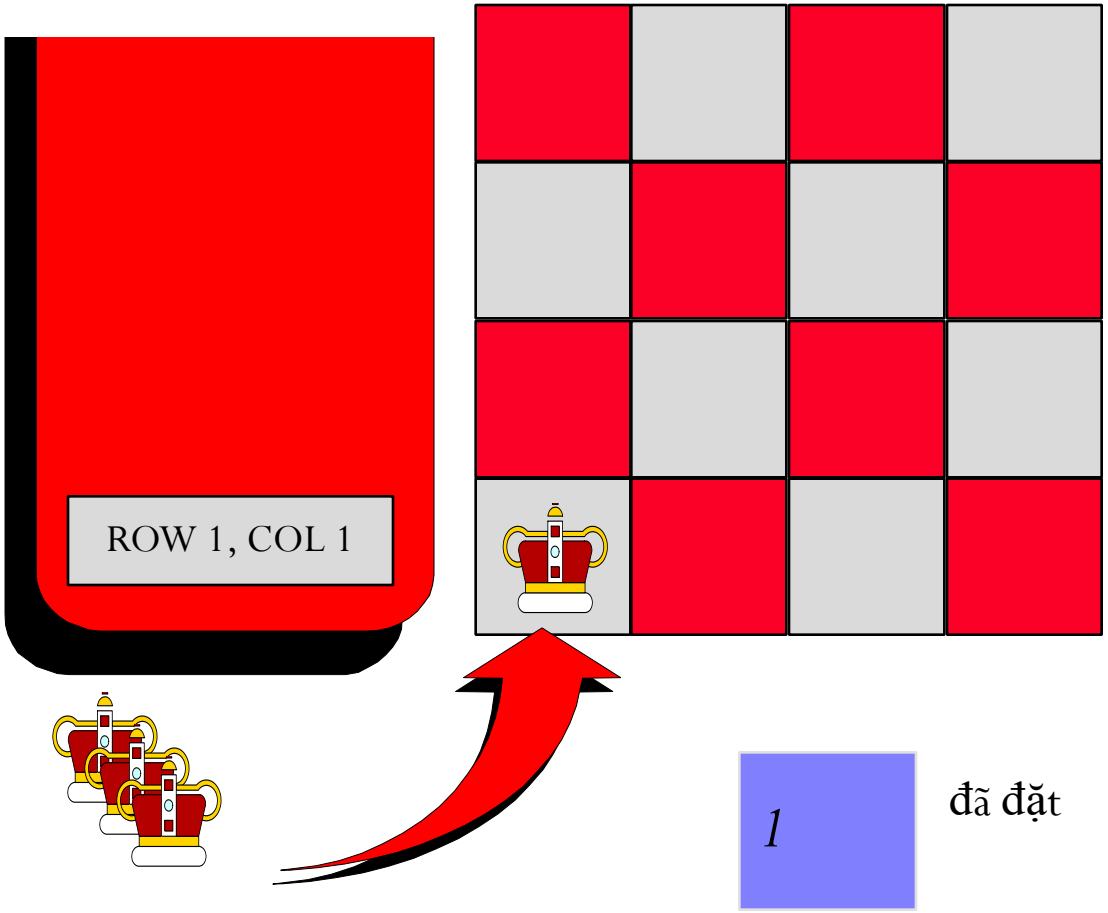


Thuật toán làm việc như thế nào



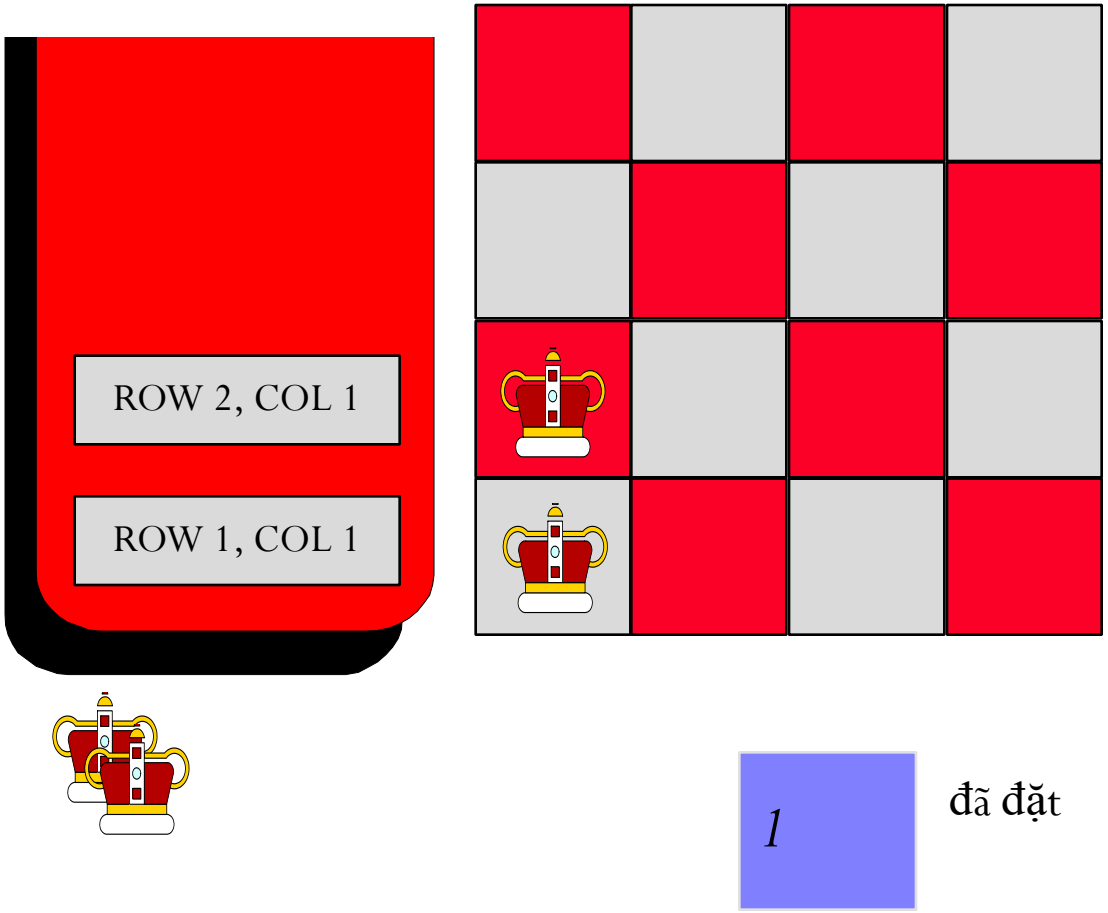
Thuật toán làm việc như thế nào

- Xếp con hậu ở dòng 1 vào vị trí cột 1



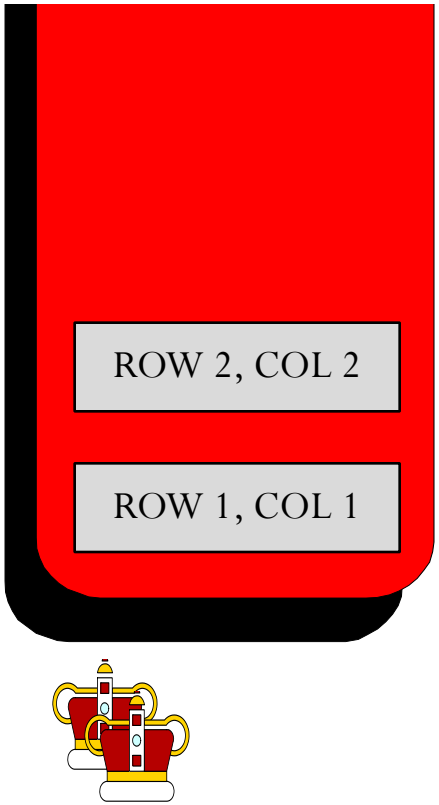
Thuật toán làm việc như thế nào

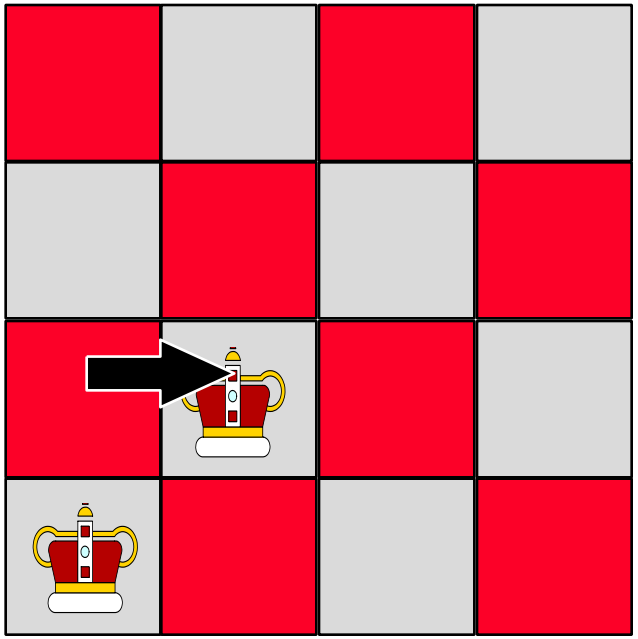
- Thử xếp con hậu ở dòng 2 vào vị trí cột 1



Thuật toán làm việc như thế nào

- Thử xếp con hậu ở dòng 2 vào vị trí cột 2

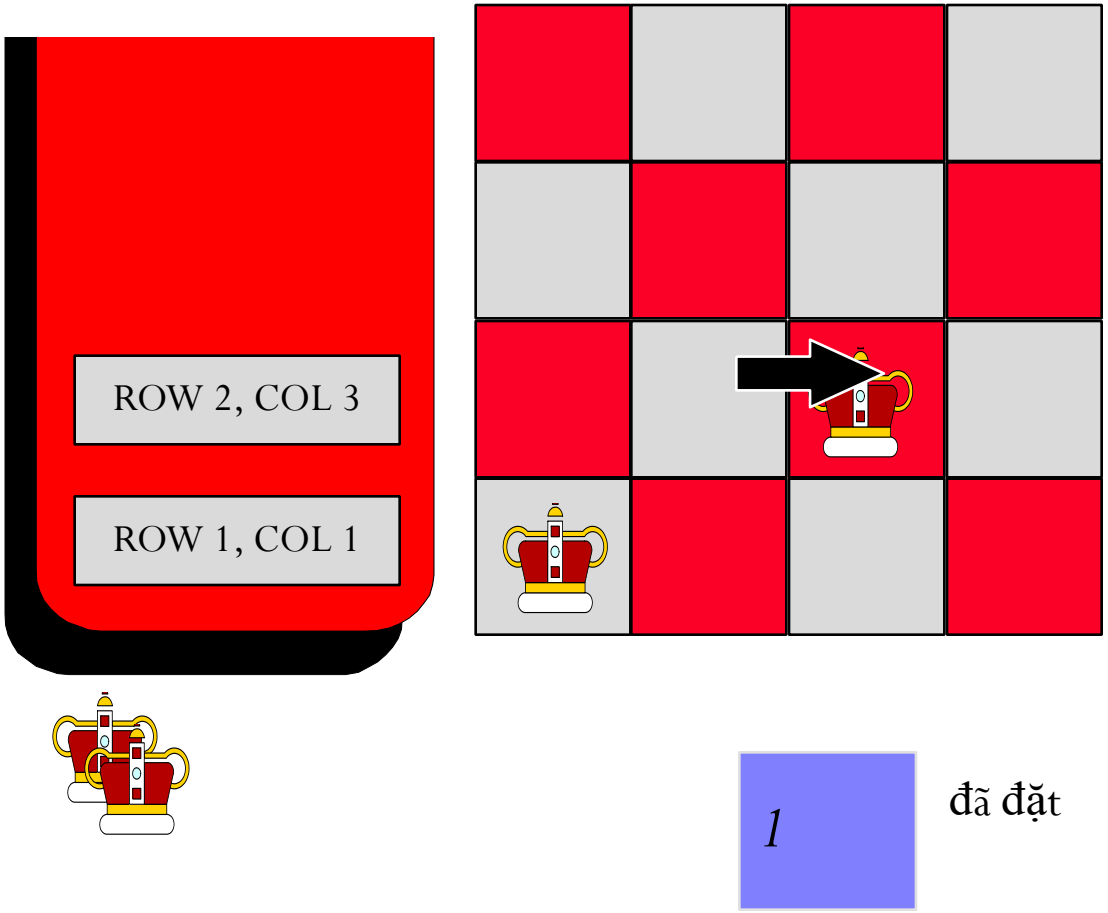




đã đặt

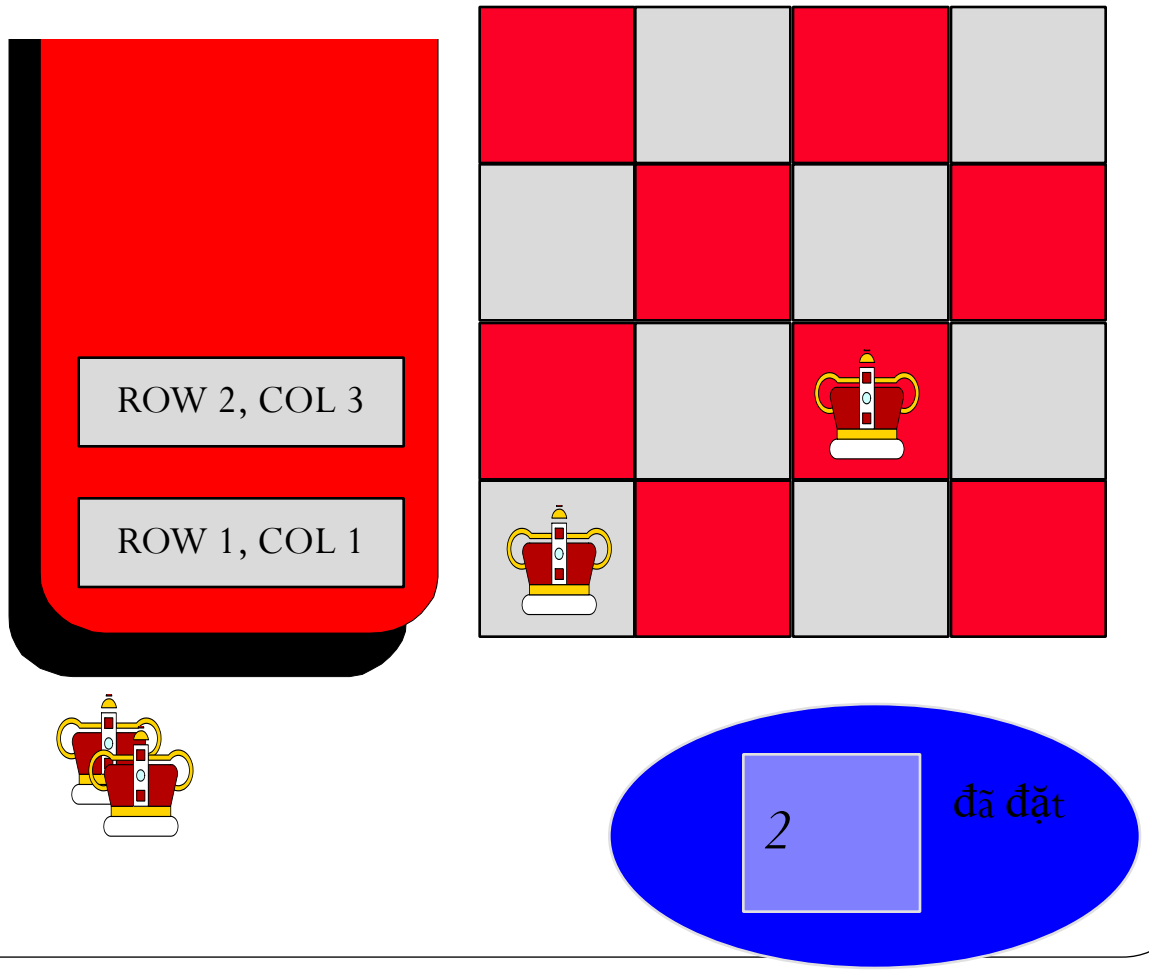
Thuật toán làm việc như thế nào

- Thử xếp con hậu ở dòng 2 vào vị trí cột 3



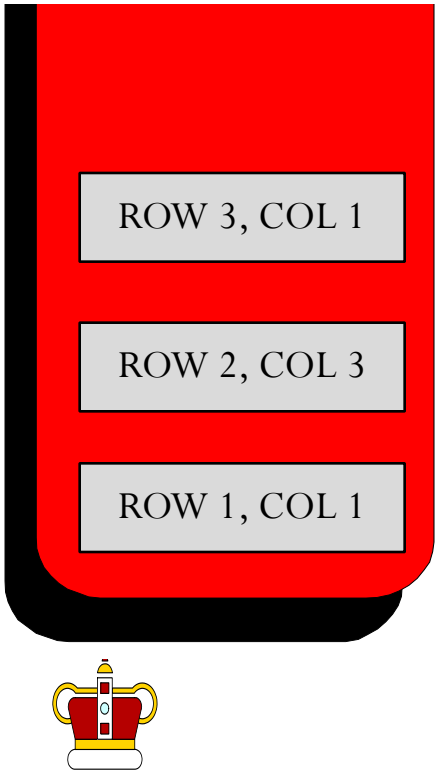
Thuật toán làm việc như thế nào

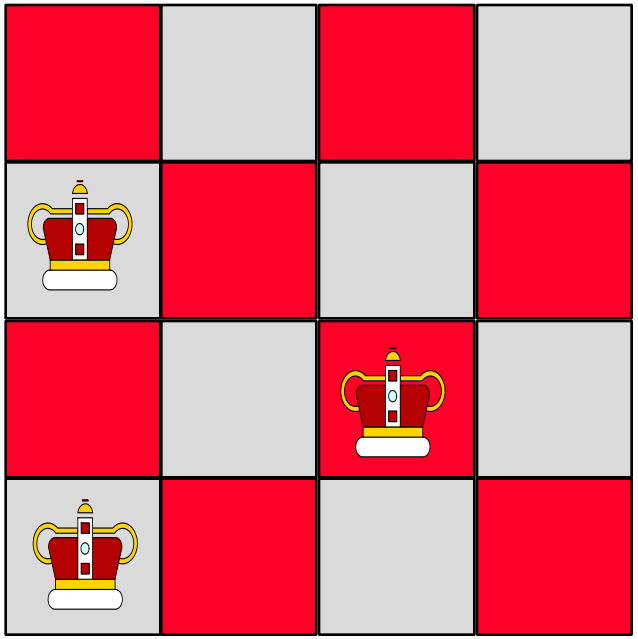
- Chấp nhận xếp con hậu ở dòng 2 vào vị trí cột 3



Thuật toán làm việc như thế nào

Thử xếp con hậu ở
dòng 3
vào cột đầu tiên

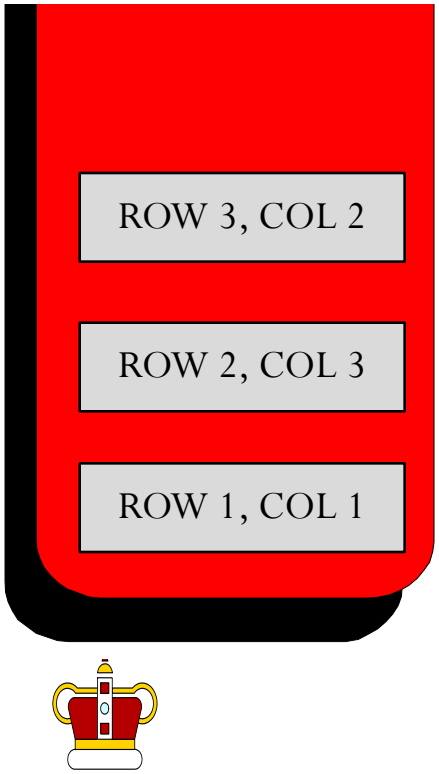


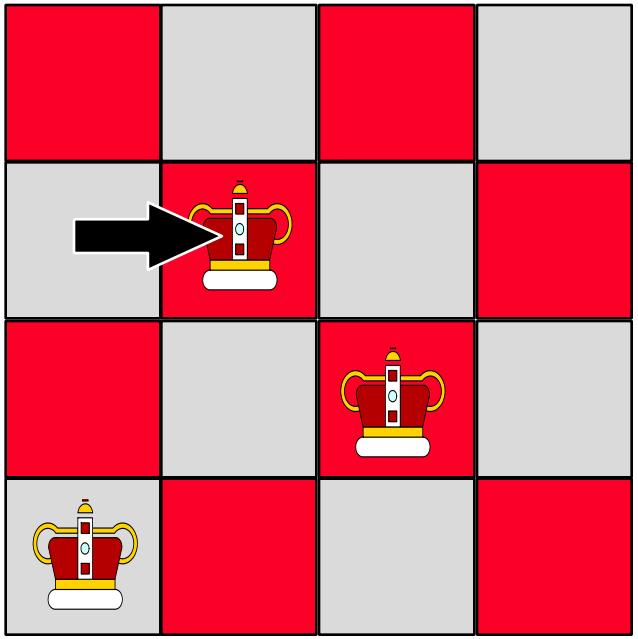


2 đã đặt

Thuật toán làm việc như thế nào

Thử cột tiếp theo

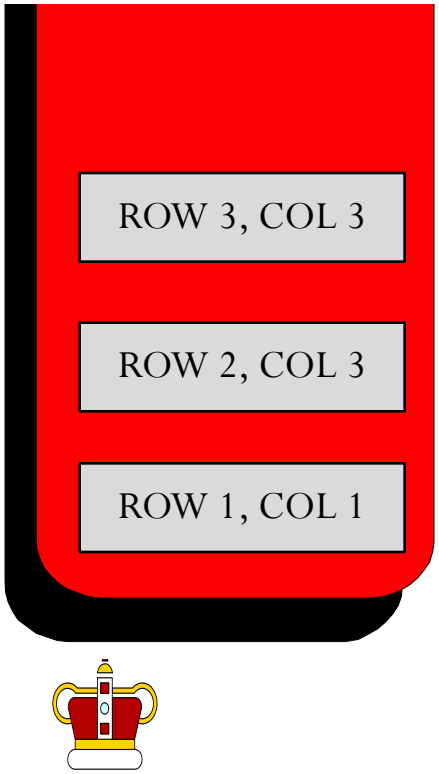


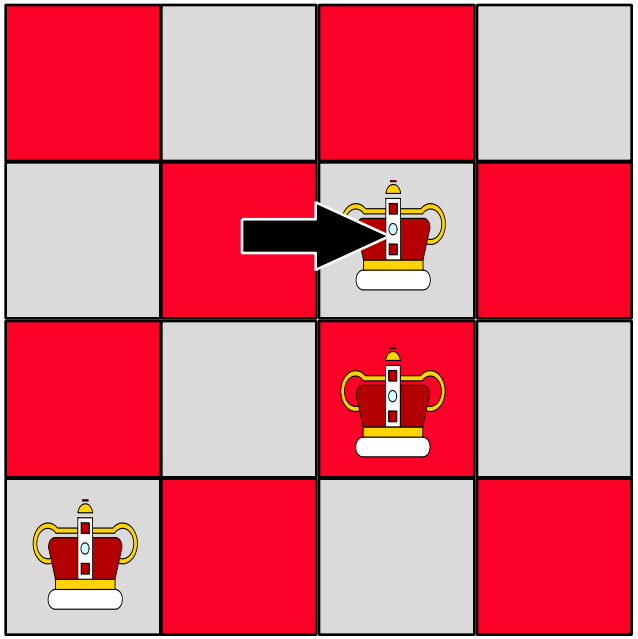


2 đã đặt

Thuật toán làm việc như thế nào

- Thử cột tiếp theo

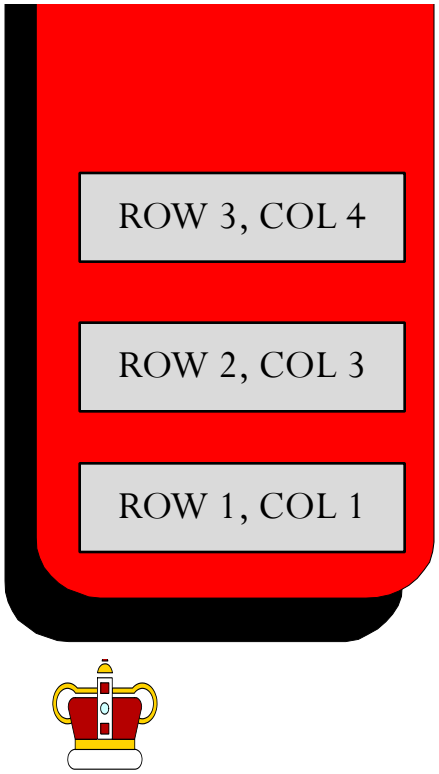


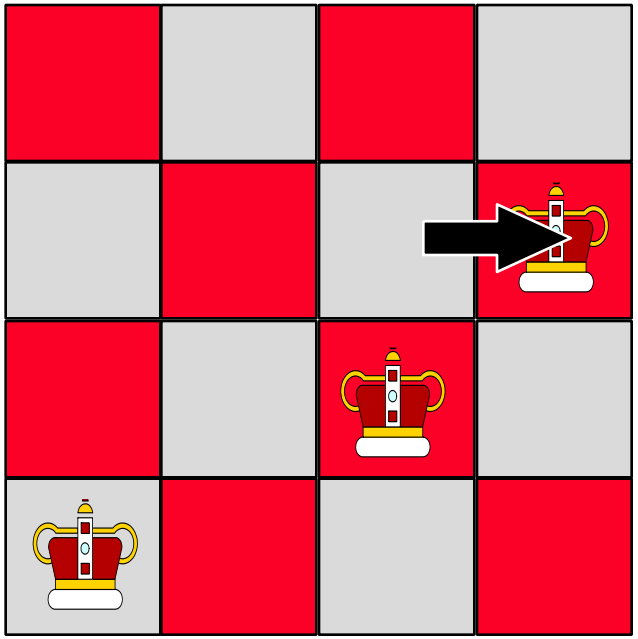


2 đã đặt

Thuật toán làm việc như thế nào

- Thử cột tiếp theo

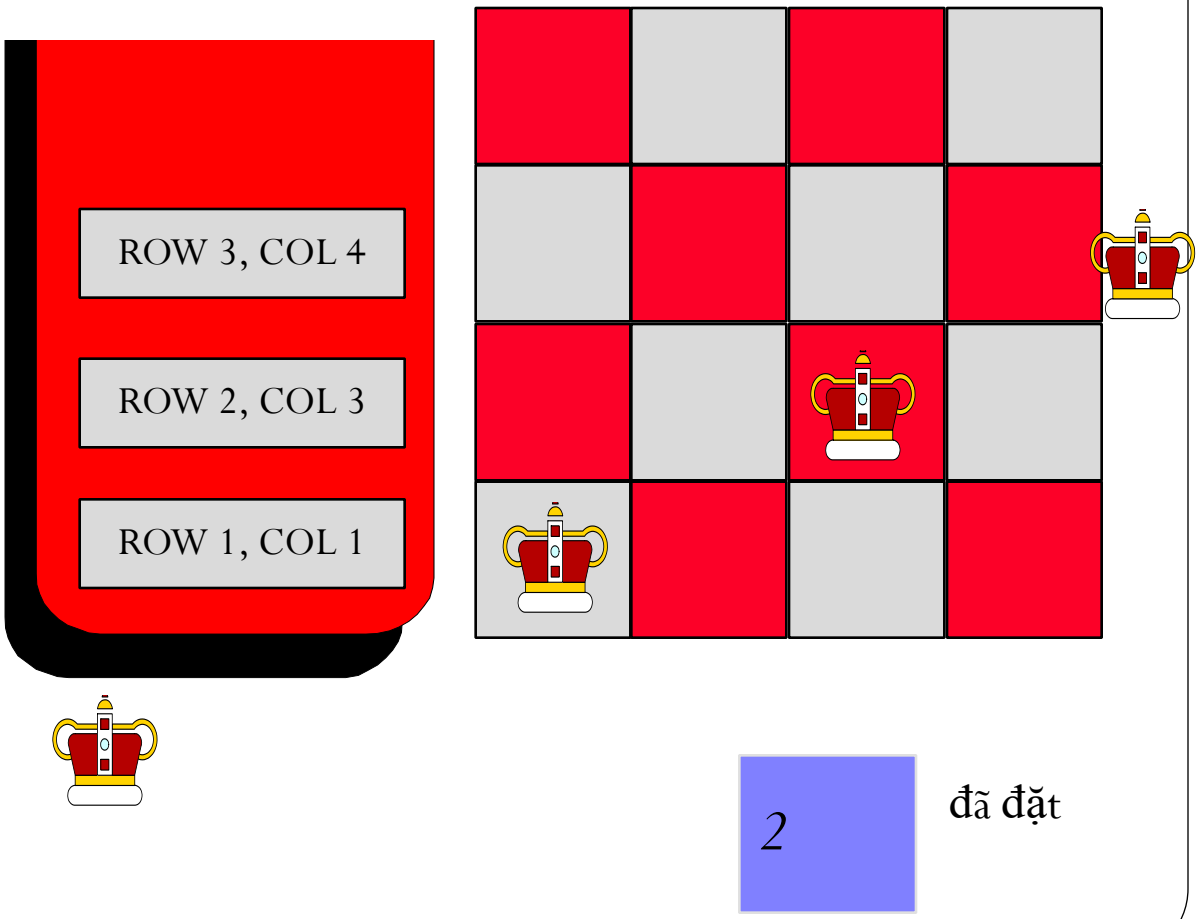




2 đã đặt

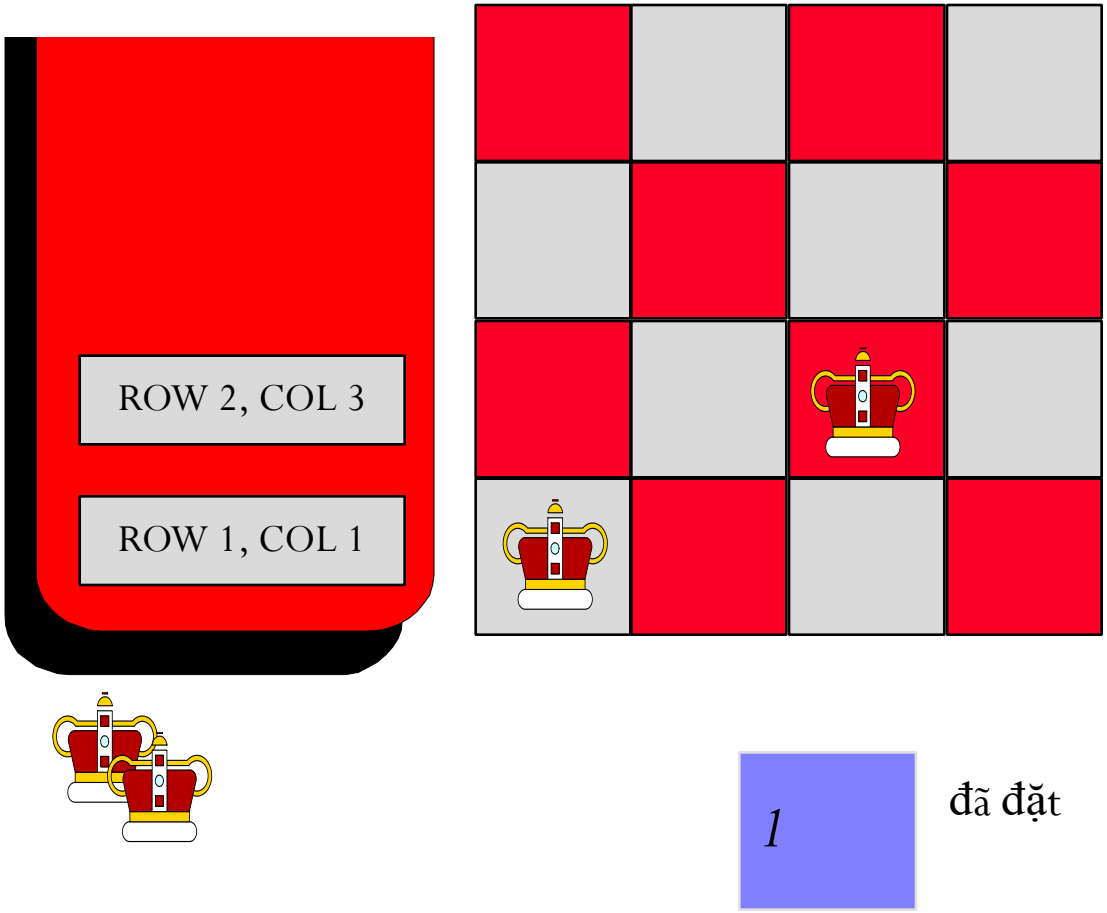
Thuật toán làm việc như thế nào

...không có vị trí đặt con hậu ở dòng 3.



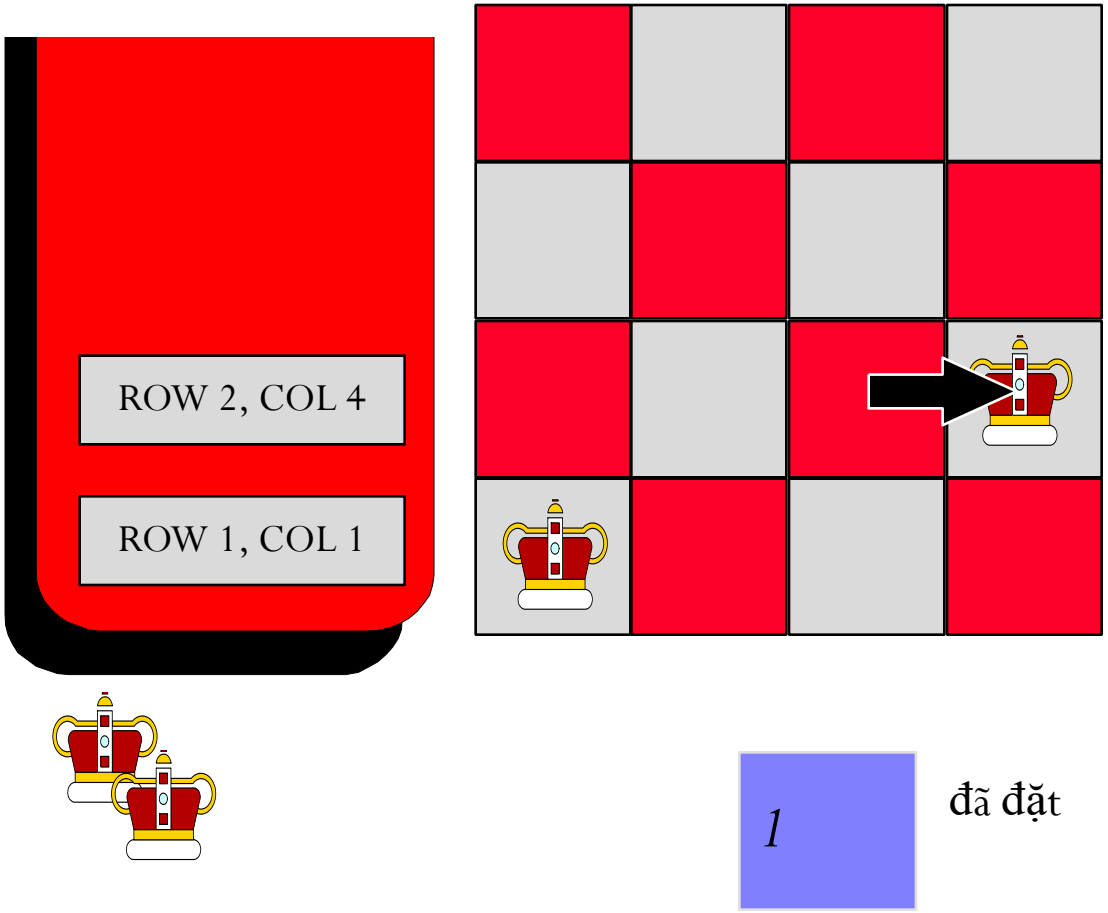
Thuật toán làm việc như thế nào

Quay lại dịch chuyển con hậu ở dòng 2



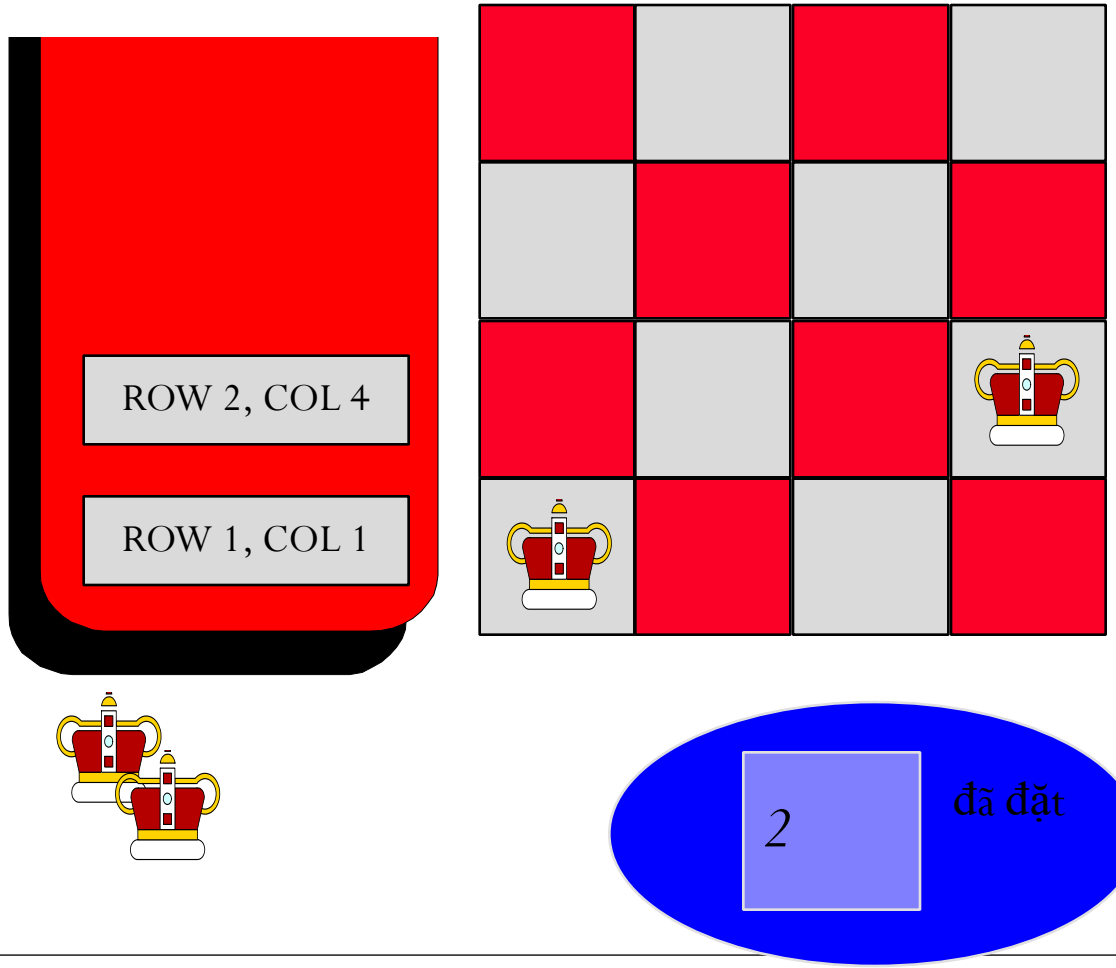
Thuật toán làm việc như thế nào

Đẩy con hậu ở dòng 2 sang cột thứ 4.



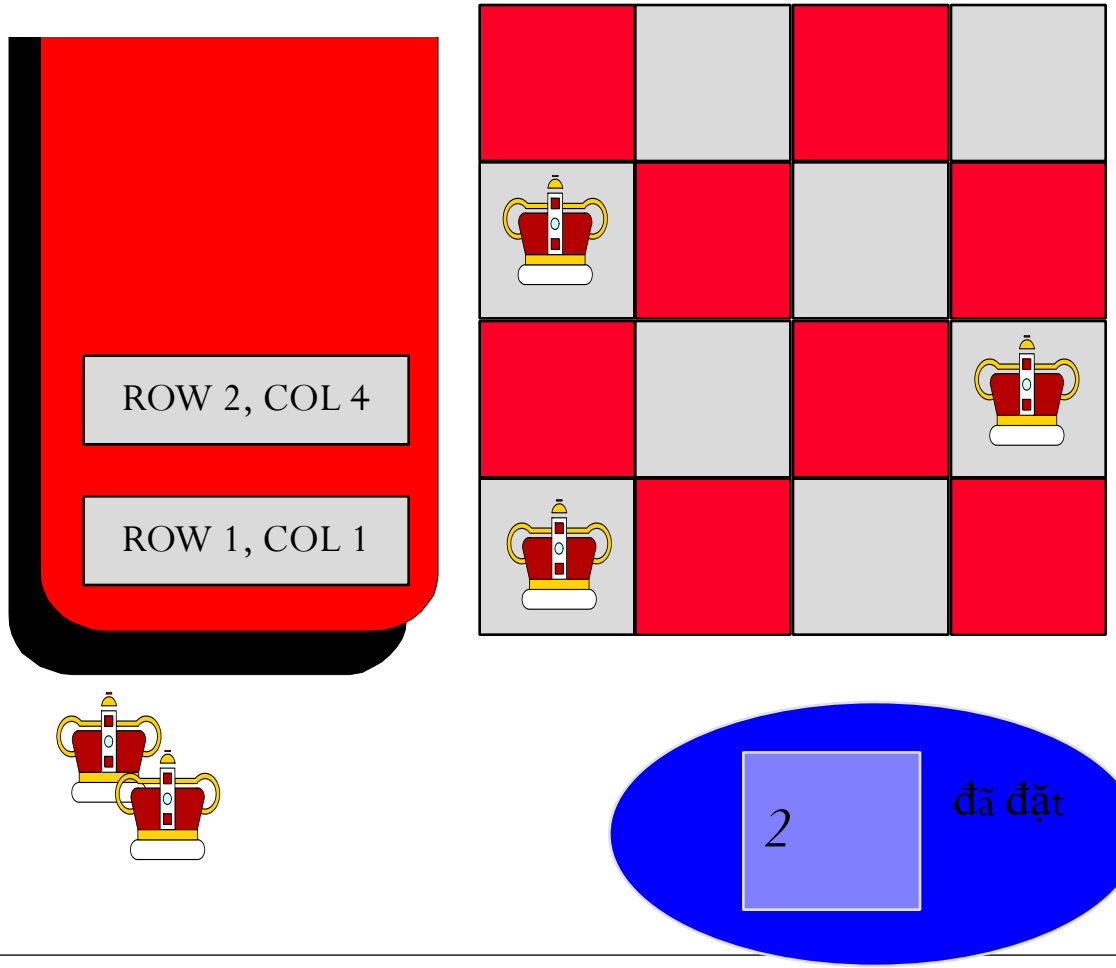
Thuật toán làm việc như thế nào

Xếp được con hậu ở dòng 2 ta tiếp tục xếp con hậu ở dòng 3



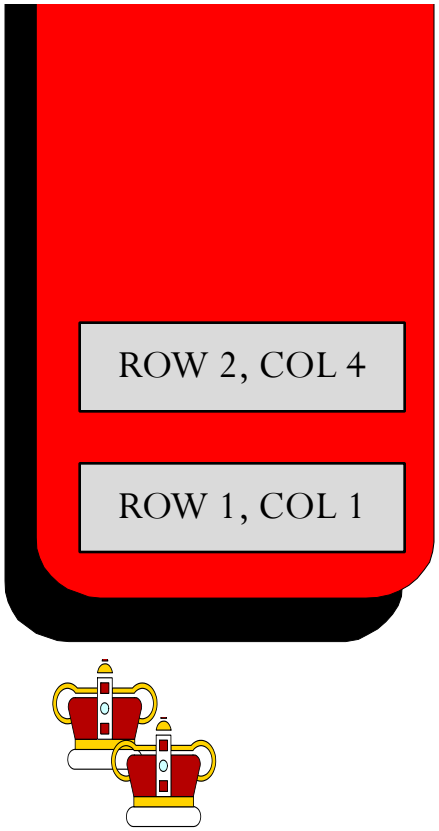
Thuật toán làm việc như thế nào

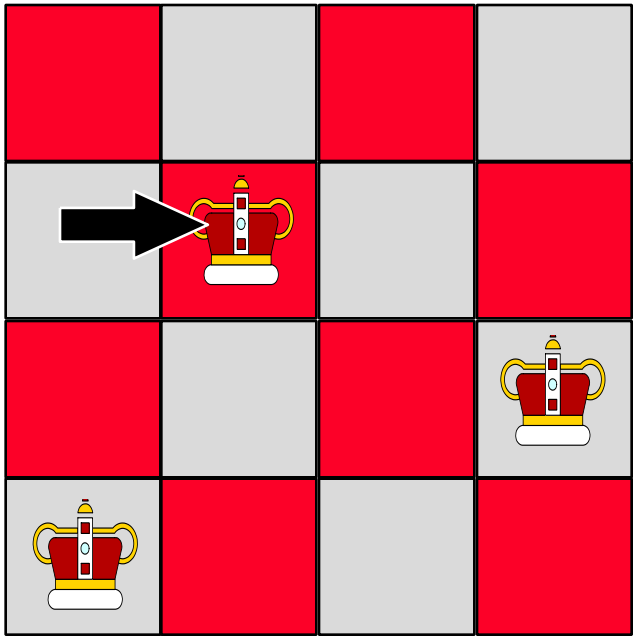
Thử xếp con hậu ở dòng 3 vào cột 1

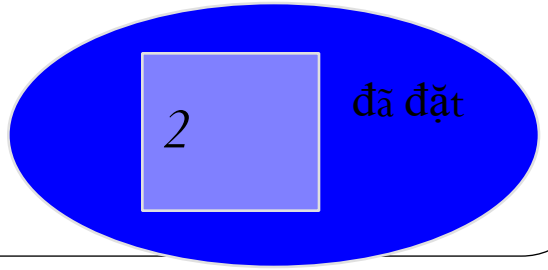


Thuật toán làm việc như thế nào

Thử xếp con hậu ở dòng 3 vào cột 2

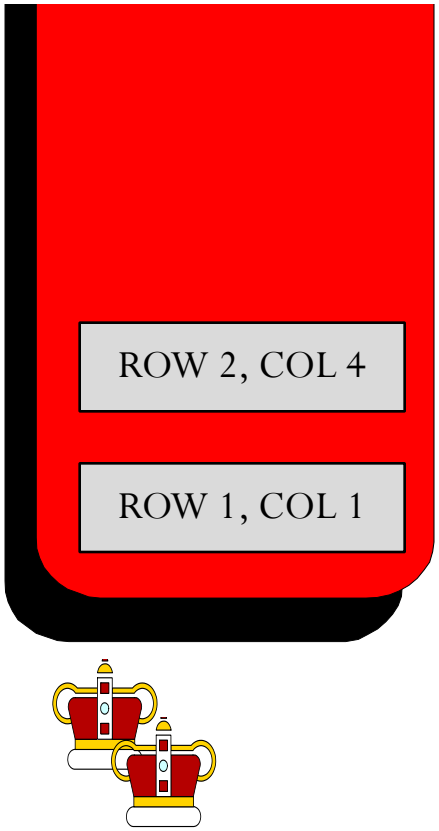


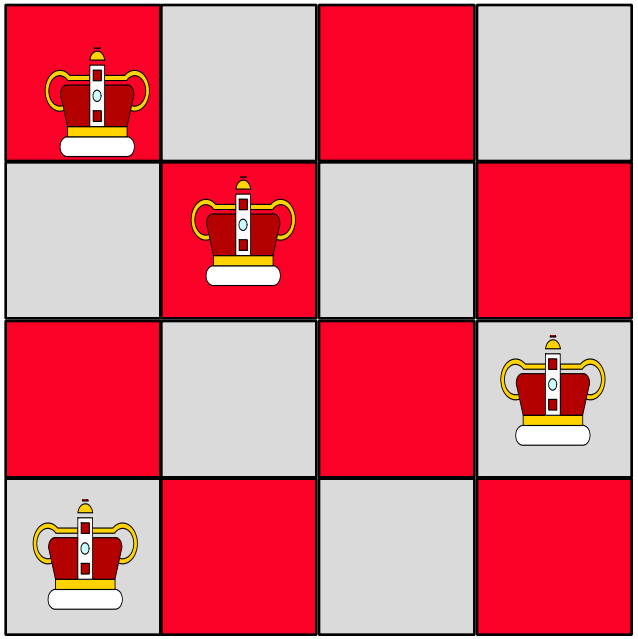


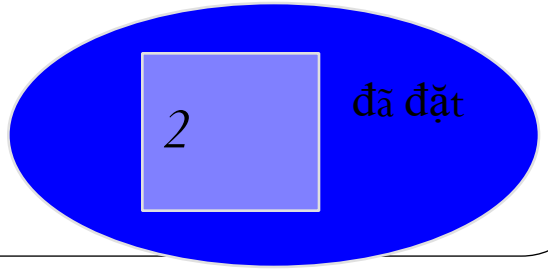


Thuật toán làm việc như thế nào

Xếp được con hậu ở dòng 3 ta tiếp tục xếp con hậu ở dòng 4: Thử cột 1

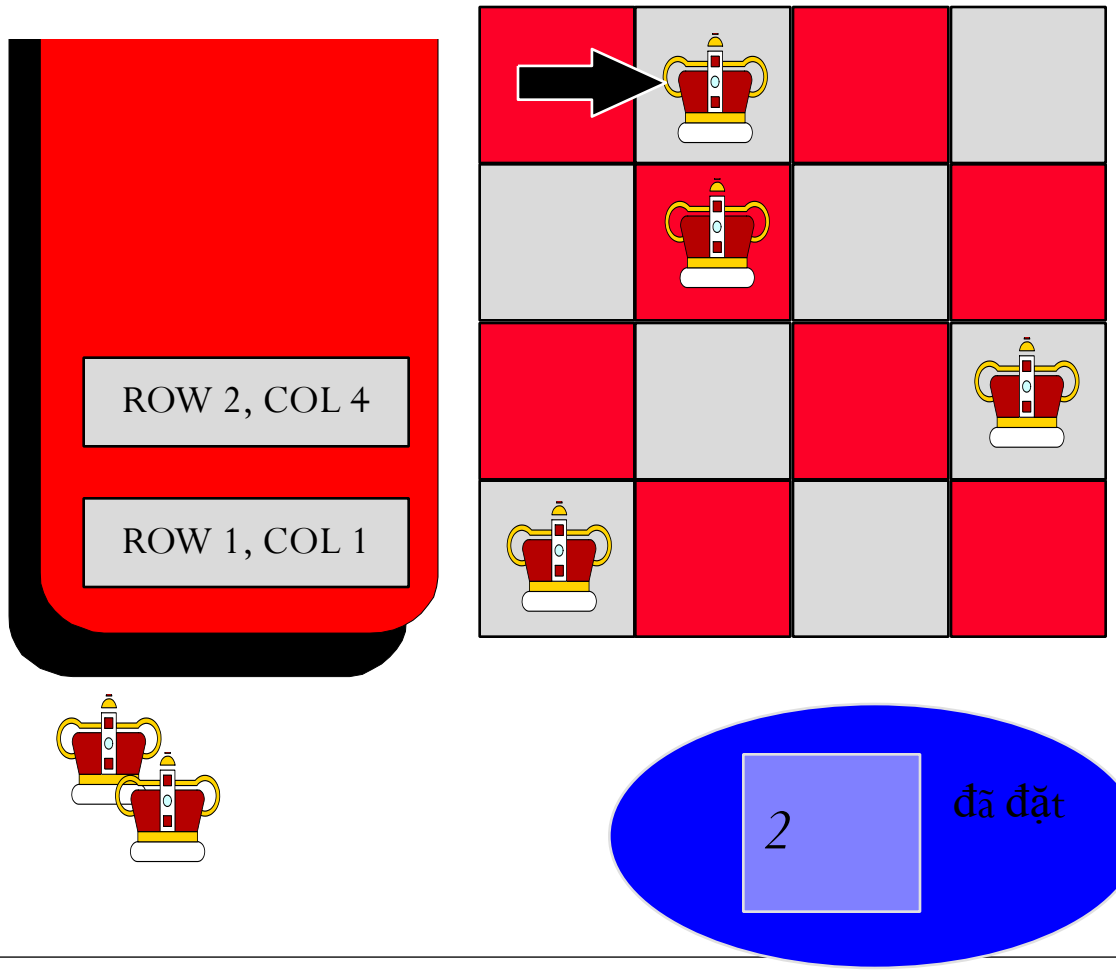






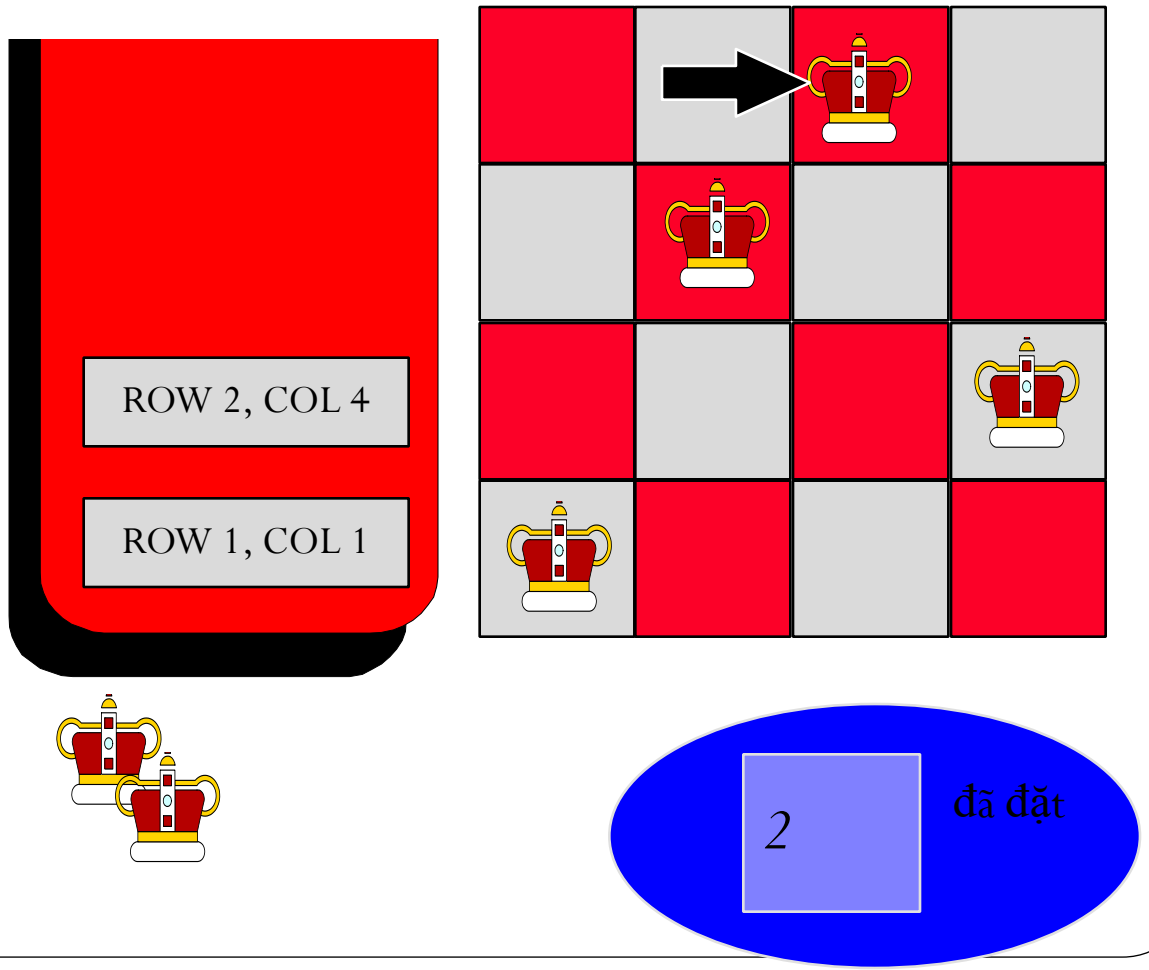
Thuật toán làm việc như thế nào

Thử xếp được con hậu ở dòng 4 vào cột 2



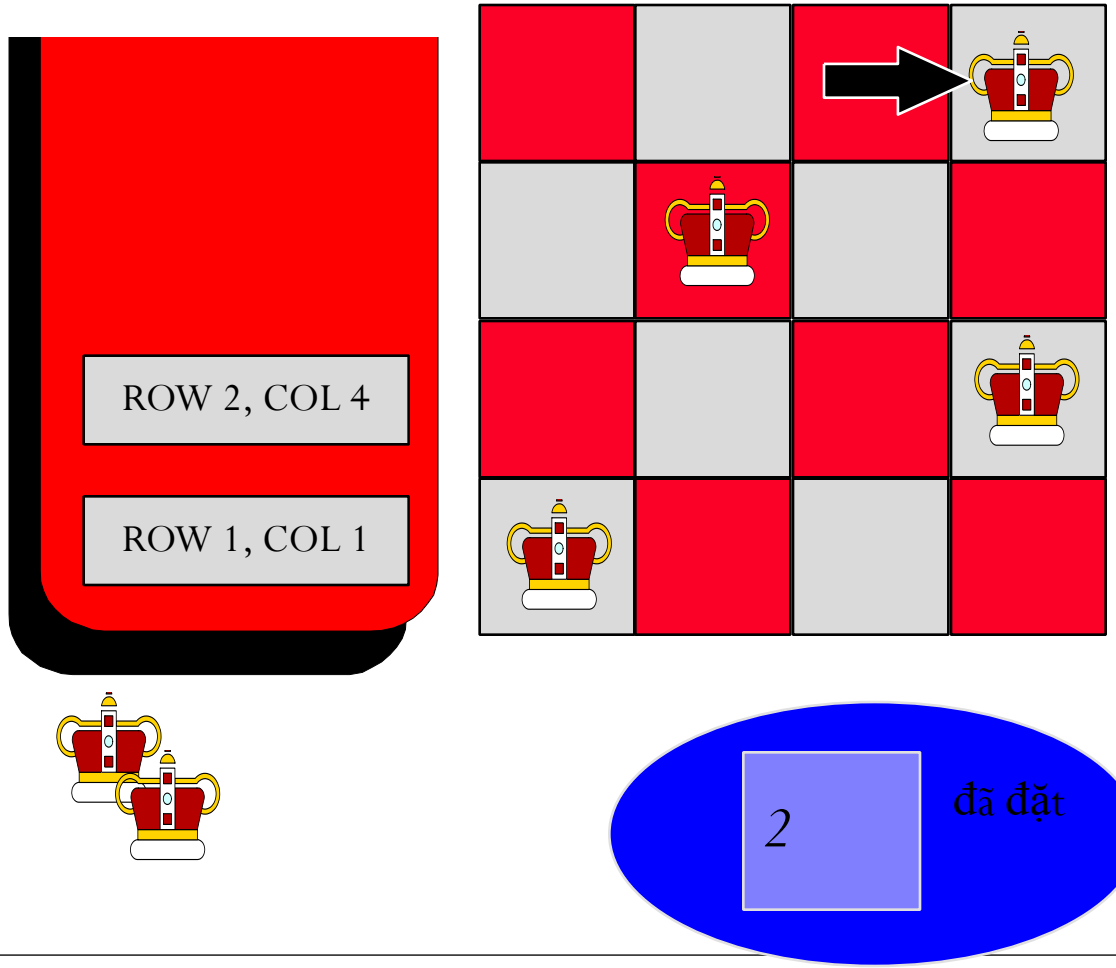
Thuật toán làm việc như thế nào

Thử xếp được con hậu ở dòng 4 vào cột 3



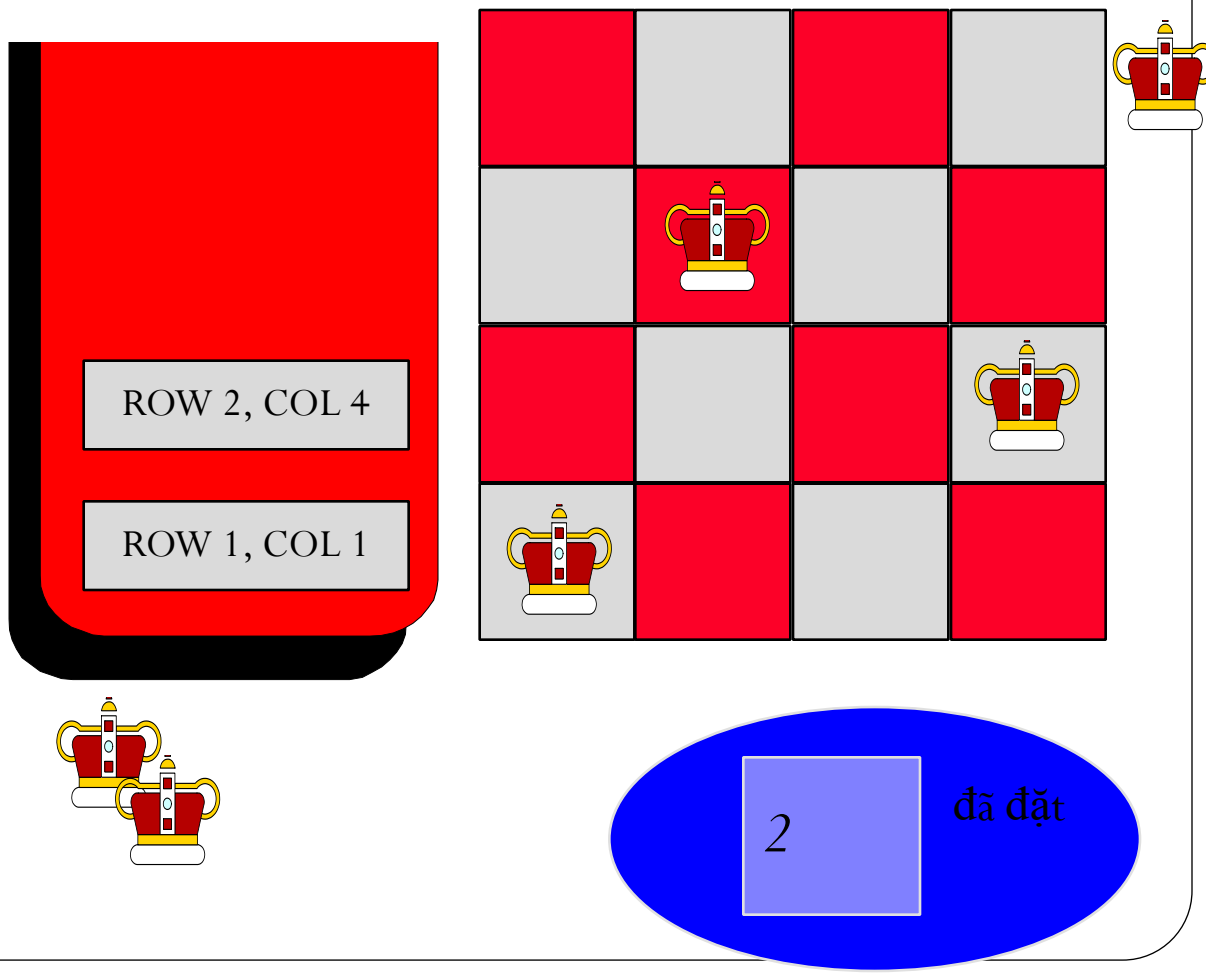
Thuật toán làm việc như thế nào

Thử xếp được con hậu ở dòng 4 vào cột 4



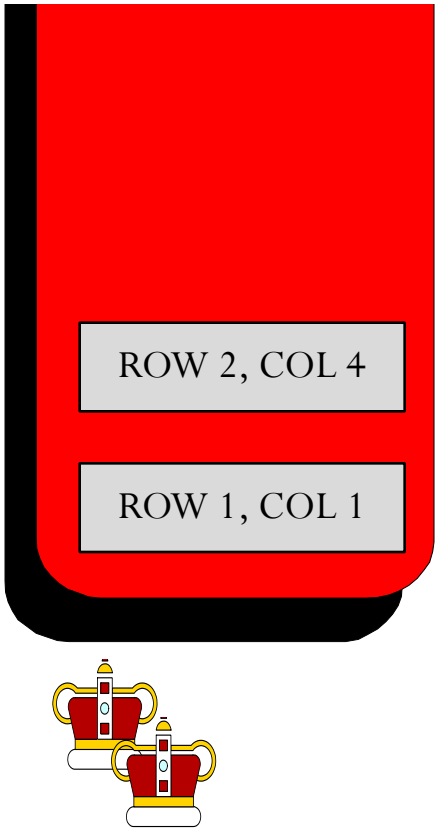
Thuật toán làm việc như thế nào

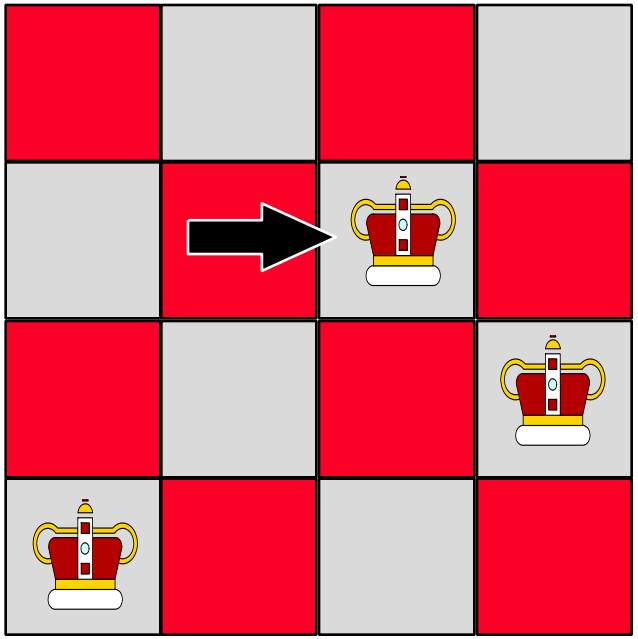
Không xếp được con hậu ở dòng 4

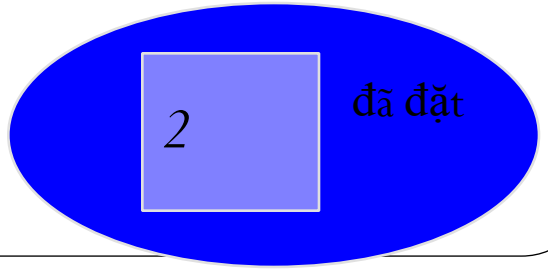


Thuật toán làm việc như thế nào

Quay lại tìm vị trí mới cho con hậu ở dòng 3: Thử cột 3

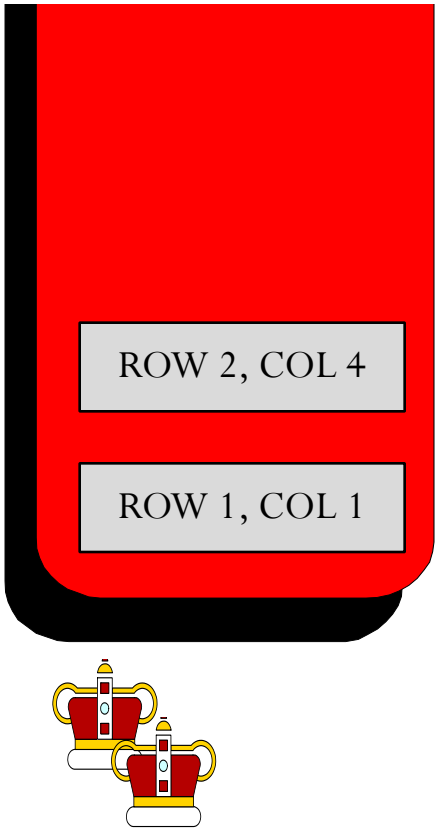


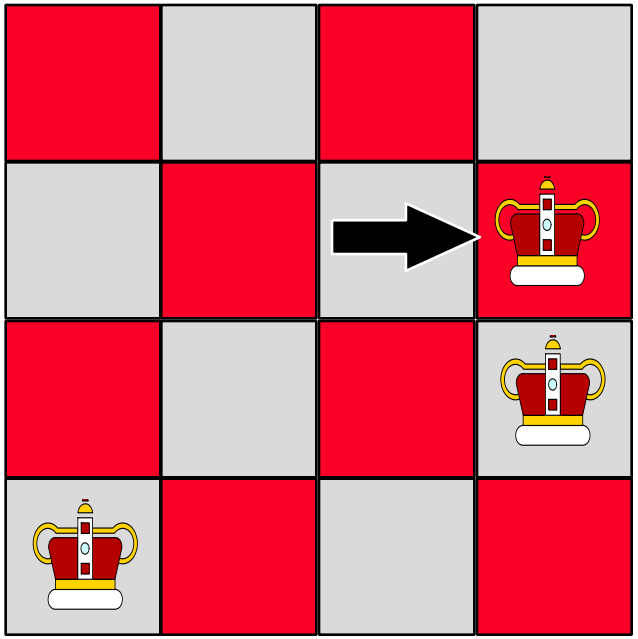


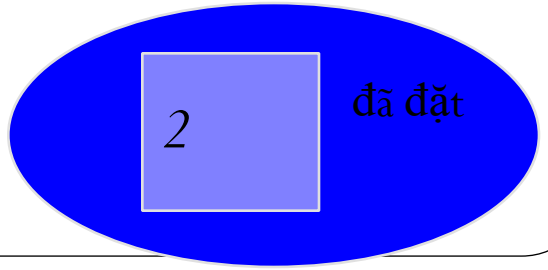


Thuật toán làm việc như thế nào

Quay lại tìm vị trí mới cho con hậu ở dòng 3: Thử cột 4

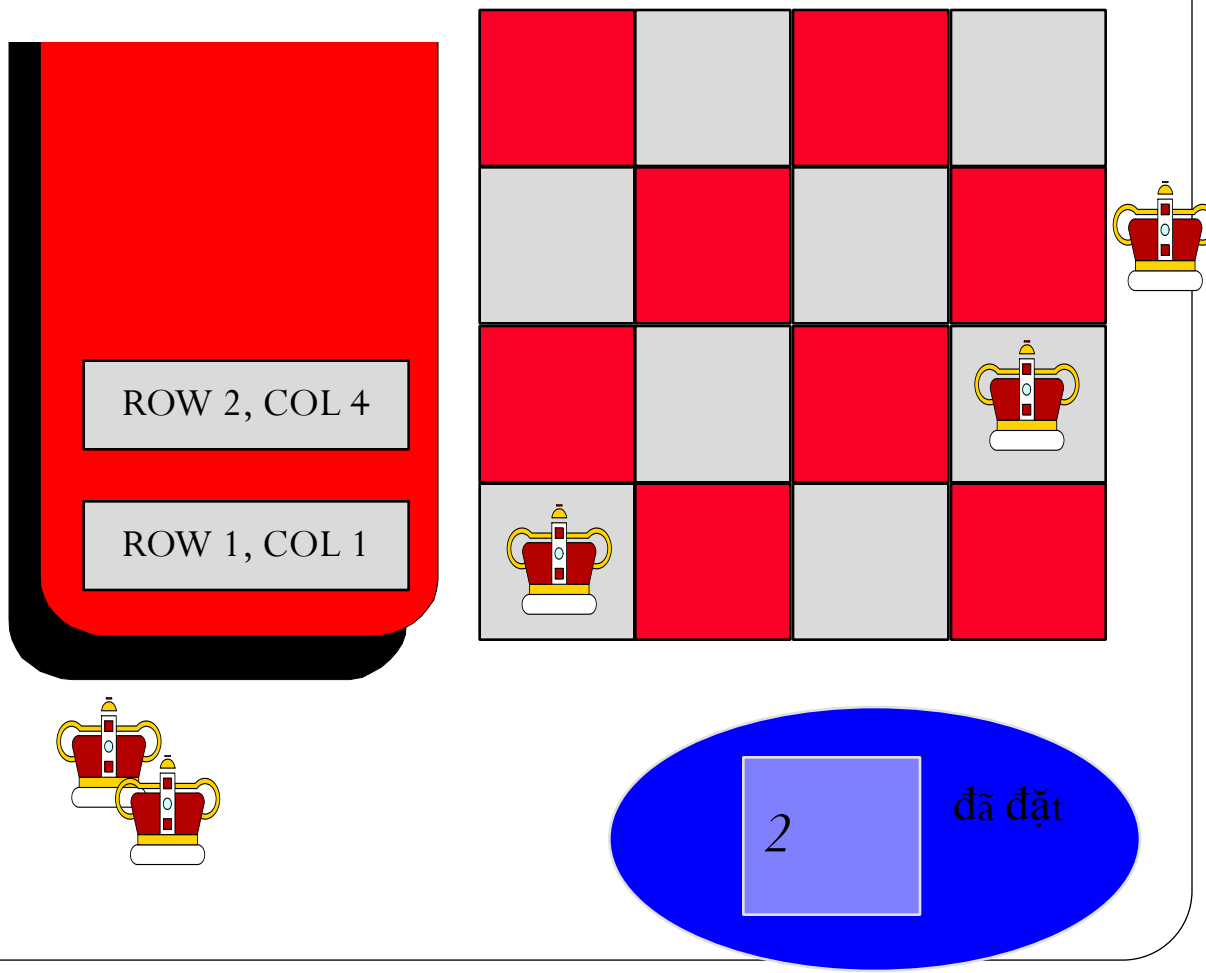






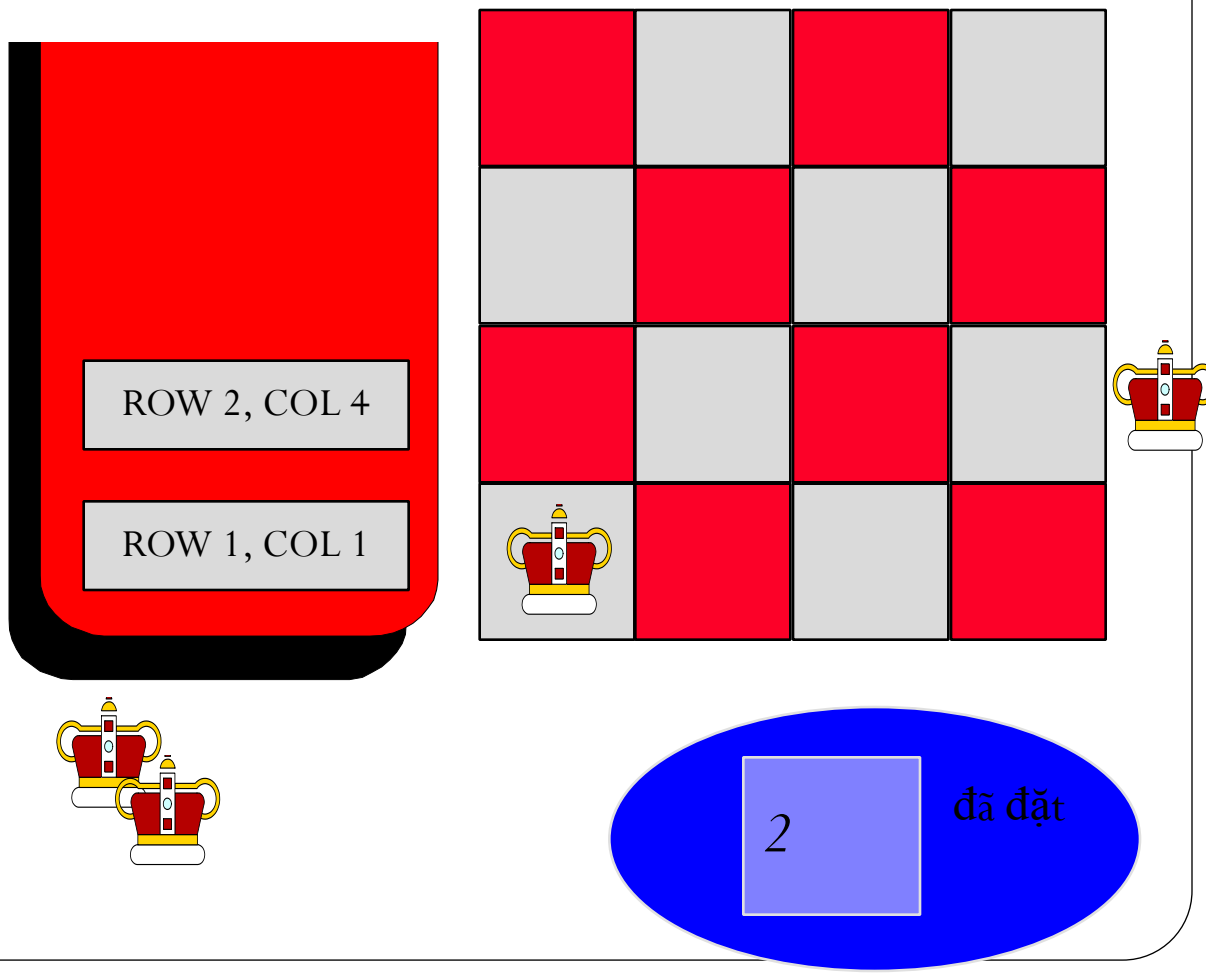
Thuật toán làm việc như thế nào

Không có cách xếp mới cho con hậu ở dòng 3



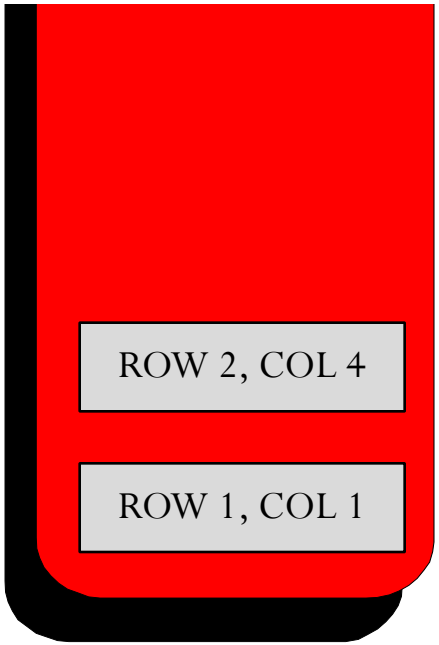
Thuật toán làm việc như thế nào

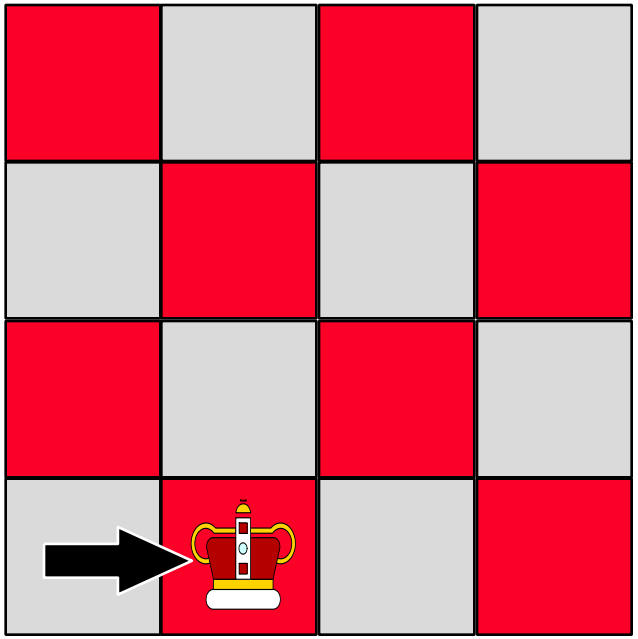
Quay lại tìm cách xếp mới cho con hậu ở dòng 2: Không có

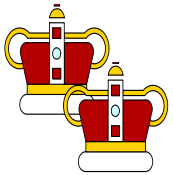


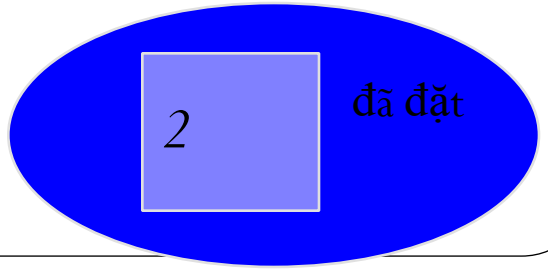
Thuật toán làm việc như thế nào

Quay lại tìm cách xếp mới cho con hậu ở dòng 1: Chuyển sang cột 2

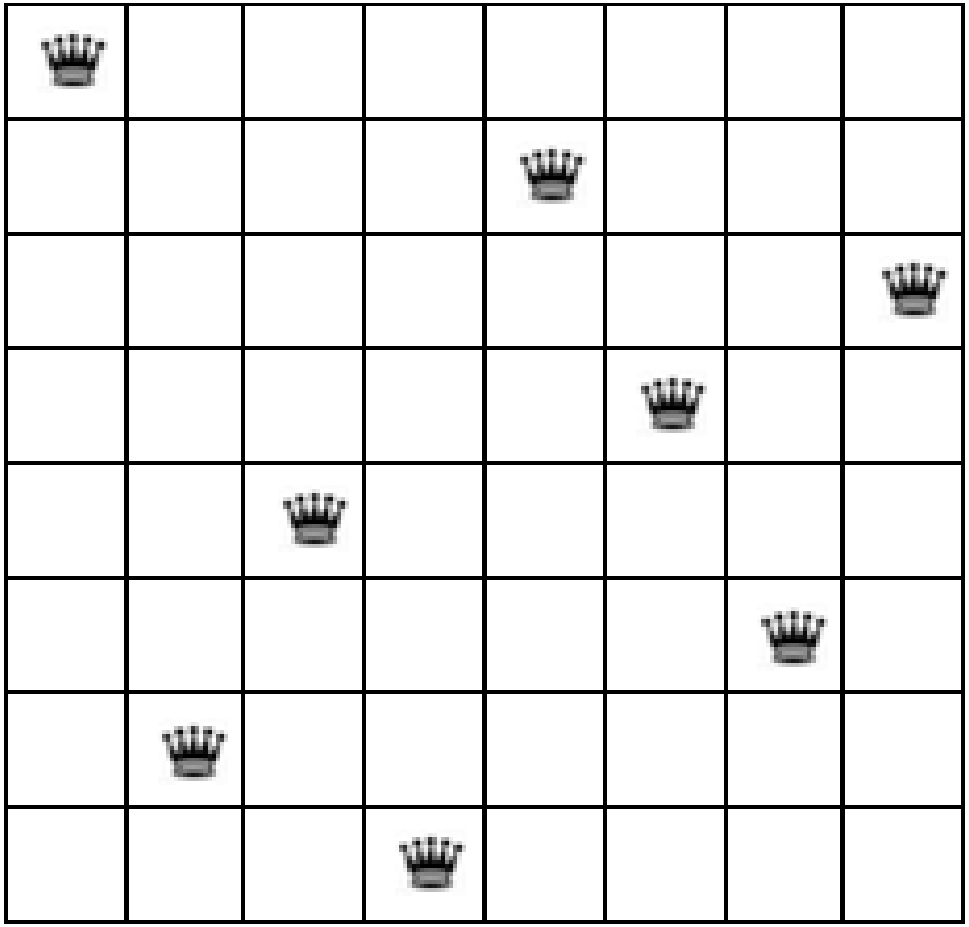








Một lời giải của bài toán xếp hậu khi $n = 8$



3. CHIA ĐỂ TRỊ

Divide and Conquer

Phương pháp

- Tư tưởng của chiến lược chia để trị như sau:
  - Người ta phân bài toán cần giải thành các bài toán con.
  - Các bài toán con lại được tiếp tục phân thành các bài toán con nhỏ hơn, cứ thế tiếp tục cho tới khi ta nhận được các bài toán con hoặc đã có thuật giải hoặc là có thể dễ ràng đưa ra thuật giải.
  - Sau đó ta tìm cách kết hợp các nghiệm của các bài toán con để nhận được nghiệm của bài toán con lớn hơn, để cuối cùng nhận được nghiệm của bài toán cần giải

Phương pháp

- Thuật toán chia ñể trị có thể biểu diễn bằng mô hình ñệ quy như sau:

```
procedure DivideConquer(A,x);
// tìm nghiệm x của bài toán A
begin
        if (A ñủ nhỏ) then Solve(A)
        else
        begin
                Phân A thành các bài toán con A1, A2, …,
                Am;
                for i:=1 to m do DivideConquer(Ai, xi);
                        Kết hợp các nghiệm xi (i=1,2,..,m)
                        của các bài toán con Ai để nhận
                        được nghiệm của bài toán A;
        end;
end;
```

Bài toán tính a^n

Bài toán: Cho số a và số nguyên dương n, tính a^n

- Cách 1: Sử dụng thuật toán lặp, mất n phép nhân để tính a^n

```
procedure power(a,n:longint;var p:longint);
{giá trị an sẽ ñược lưu vào biến p}
var i : longint;
begin
      p:=1;
      for i:=1 to n do p:=p*a;
end;
var a, n, p : longint;
BEGIN
write('Nhap a, n:'); readln(a, n);
power(a,n,p);
write(p);
END.
```

Bài toán tính a^n

- Cách 2: Áp dụng kĩ thuật chia để trị, ta tính a^n dựa vào a^k (trong đó k = n div 2) như sau:
  - Nếu n chẵn: $a^n = a^k \times a^k$
  - Nếu n lẻ: $a^n = a^k \times a^k \times a$

Để tính a^k ta lại dựa vào $a^{k \text{ div } 2}$ quá trình chia nhỏ cho đến khi nhận ñược bài toán tính a^1 thì dừng

Ví dụ: Tính 9^{13}

- Bài toán được tính dựa trên bài toán con 9^6, ta có $9^{13} = 9^6 \times 9^7$
- Bài toán 9^6 được tính dựa vào bài toán con 9^3, ta có $9^6 = 9^3 x 9^3$
- Bài toán 9^3 được tính dựa vào bài toán con 9^1, ta có $9^3 = 9^1 x 9^1 x 9^1$

Bài toán tính a^n

- Thủ tục ñệ quy power(a,n,p) sau thể hiện ý tưởng trên

```
procedure power(a,n:longint; var p:longint);
var tmp : longint;
begin
        if (n=1) then p:=a
        else begin
                power(a,n div 2,tmp);
                if (n mod 2=1) then p:=tmp*tmp*a
                else p:=tmp*tmp;
        end;
end;
```

Bài toán tính a^n

- hoặc viết dưới dạng hàm như sau:

```
function power(a,n:longint):longint;
var tmp : longint;
begin
        if (n=1) then exit(a)
        else begin
                tmp:=power(a,n div 2);
                if (n mod 2=1) then exit(tmp*tmp*a)
                else exit(tmp*tmp);
        end;
end;
```

- Thuật toán chia để trị mất không quá *2logn* phép nhân, nhỏ hơn rất nhiều so với *n* phép nhân

4. QUY HOẠCH ĐỘNG

Dynamic programming

PHƯƠNG PHÁP

- Tư tưởng cơ bản của phương pháp quy hoạch động là sử dụng một bảng để lưu giữ lời giải của các bài toán con đã được giải.
- Khi giải một bài toán con cần đến nghiệm của bài toán con cỡ nhỏ hơn, ta chỉ cần lấy lời giải ở trong bảng mà không cần phải giải lại. Chính vì thế mà các thuật toán được thiết kế bằng quy hoạch động sẽ rất hiệu quả.

PHƯƠNG PHÁP

- Để giải quyết một bài toán bằng phương pháp quy hoạch động, chúng ta cần tiến hành những công việc sau:
  - Tìm nghiệm của các bài toán con nhỏ nhất.
  - Tìm ra công thức (hoặc quy tắc) xây dựng nghiệm của bài toán con thông qua nghiệm của các bài toán con cỡ nhỏ hơn.
  - Tạo ra một bảng lưu giữ các nghiệm của các bài toán con. Sau ñó tính nghiệm của các bài toán con theo công thức đã tìm ra và lưu vào bảng.
  - Từ các bài toán con đã giải để tìm nghiệm của bài toán.

Tìm số Fibonacci

- Ví dụ: Tìm số Fibonacci sử dụng phương pháp quy hoạch động

Ta sử dụng mảng S[0..,Max], S[i] để lưu lại lời giải cho bài toán tính số Fibonacci thứ i

```
#include <stdio.h>
#define MAX 50
int s[MAX], n, k;
int F(int n)
{
        if (s[n] == -1)
        {       //bài toán chua ñu?c gi?i thì s? ti?n hành gi?i
                if(n==1||n==0) s[n]=1;
                else s[n] = F(n-1) + F(n-2);
        }
        //n?u bài toán ñã ñu?c gi?i thì không c?n gi?i n?a mà l?y luôn k?t qu?
        return s[n];
}

int main()
{
        printf("Nhap n:");
        scanf("%d",&n);
        for (k=0;k<MAX;k++) s[k]=-1;
        printf("%d", F(n));
}
```

Tìm số Fibonacci

- Ta cũng có thể cài đặt phương pháp quy hoạch động cho bài toán như sau:

```
#include <stdio.h>
#define MAX 50
int s[MAX], n, k;
int main()
{
        printf("Nhap n:");
        scanf("%d",&n);
        s[0]=1;s[1]=1;
        for (k=2;k<=n;k++) s[k]=s[k-1]+s[k-2];
        printf("%d", s[n]);
}
```

Dãy con chung dài nhất

- Cho hai số nguyên dương M, N (0<M, N < 100) và hai dãy số nguyên: $A_1, A_2,..., A_M$ và $B_1, B_2,..., B_N$. Tìm một dãy dài nhất C là dãy con chung dài nhất của hai dãy A và B, nhận được từ A bằng cách xoá đi một số số hạng và cũng nhận được từ B bằng cách xoá đi một số số hạng
- Dữ liệu vào trong file LCS.INP có dạng:
  - Dòng thứ nhất chứa 1 số M
  - Dòng thứ hai chứa M số $A_1, A_2,..., A_M$
  - Dòng thứ ba chứa 1 số N.
  - Dòng thứ tư chứa N số $B_1, B_2,..., B_N$.
- Dữ liệu ra trong file LCS.OUT có dạng:
  - Dòng thứ nhất ghi số k là số số hạng của dãy C.
  - Dòng thứ hai chứa k số là các số hạng của dãy C.

Dãy con chung dài nhất

- Giải
  - ✓ Cần xây dựng mảng L[0..M, 0..N] với ý nghĩa: L[i, j] là độ dài của dãy chung dài nhất của hai dãy A[0.. i] và B[0..j].
  - ✓ đương nhiên nếu một dãy là rỗng (số phần tử là 0) thì dãy con chung cũng là rỗng vì vậy

    $$L[0, j] = 0\ \forall j,\ j = 1..\ N,\ L[i, 0] = 0\ \forall i,\ i = 1..\ M.$$
  - ✓ Với $M \geq i > 0$ và $N \geq j > 0$ thì L[i, j] được tính theo công thức truy hồi sau:

    $$L[i,j] = Max\{L[i, j-1], L[i-1, j], L[i-1, j-1] + x\}$$

    (với $x = 0$ nếu $A[i] \neq B[j]$, $x=1$ nếu $A[i]=B[j]$)

```
#include <conio.h>
#define MAX 50
FILE *fvao, *fra;
int a[MAX],b[MAX],p[MAX],l[MAX][MAX],m,n, count;
void Enter()
{
        fvao = fopen("D:\\lcs.in","r");
        if(fvao==NULL)
        {
                printf("Loi moi tep vao");
        }
        fscanf(fvao,"%d",&m);
        for(int i=1;i<=m;i++)
                fscanf(fvao,"%d",&a[i]);
        fscanf(fvao,"%d",&n);
        for(int i=1;i<=n;i++)
                fscanf(fvao,"%d",&b[i]);
}
```

```
int max(int x, int y)
{
        return x>y?x:y;
}
void Optimize()
{
        int i, j;
        for (i=1;i<=m;i++) l[i][0] = 0;
        for (j=1;j<=n;j++) l[0][j] = 0;
        for (i=1;i<=m;i++)
                for (j=1;j<=n;j++)
                        if (a[i]==b[j])
                                l[i][j] = l[i-1][j-1]  + 1;
                        else
                                l[i][j] = max(l[i][j-1],l[i-1][j]);
}
```

```
void Trace()
{
        int i,j;
        fra = fopen("D:\\lcs.out","wt");
        if(fra==NULL)
        {printf("Loi mo tep");return;}
        fprintf(fra,"%d\n",l[m][n]);
        for(i=0;i<MAX;i++) p[i]=0;
        i = m; j = n; count = 0;
        while(i>0 && j > 0)
        {
                if (a[i]==b[j])
                {
                        count++; p[count] = a[i]; i--; j--;
                }
                else if (l[i][j]==l[i][j-1]) j--;
                else i--;
        }
        for(i=count;i>=1;i--) fprintf(fra,"%6d", p[i]);
}
```

Dãy con chung dài nhất

```
int main()
{
        Enter();
        Optimize();
        Trace();
}
```

Bài toán ba lô (knapsack)

Cho n gói hàng. Gói hàng thứ i có khối lượng là A[i] và giá trị C[i]. Cần chọn những gói hàng nào để bỏ vào một ba lô sao tổng giá trị của các gói hàng đã chọn là lớn nhất nhưng tổng khối lượng của chúng không vượt quá khối lượng M cho trước. Mỗi gói chỉ chọn 1 hoặc không chọn.

Ví dụ: n = 5; M = 13

i	1	2	3	4	5
A[i]	3	4	5	2	1
C[i]	4	5	6	3	1

Tổng giá trị của các gói hàng bỏ vào ba lô: 16
Các gói được chọn: 1(3, 4) 2(4, 5) 3(5, 6) 5(1, 1)

Tham số thể hiện kích thước bài toán

- Kết quả bài toán là tổng giá trị lớn nhất của các món hàng được chọn trong n món sao cho tổng khối lượng không lớn hơn M cho trước, ký hiệu là F(n)
- Tham số thể hiện kích thước bài toán là số món hàng n
- Giá trị của F(n) có thể được tính từ giá trị của F(n-1) cộng thêm hoặc không cộng thêm giá trị của món hàng thứ n nhưng tổng khối lượng không lớn hơn M.
- Nếu chọn thêm món hàng thứ n thì tổng khối lượng được chọn trong (n-1) món hàng không lớn hơn (M-A[n])
- Suy ra bài toán có 2 tham số: số món hàng và khối lượng giới hạn

Lập công thức đệ qui

Gọi F(i, v) là tổng giá trị lớn nhất của các gói hàng được chọn trong i gói hàng sao cho tổng khối lượng không lớn hơn v.

- Trường hợp A[i] > v:

  $$F(i, v) = F(i - 1, v)$$

- Trường hợp A[i] <= v:
  - Nếu gói hàng thứ i không được chọn thì:

    $$F(i, v) = F(i - 1, v)$$

  - Nếu gói hàng thứ i được chọn thì:

    $$F(i, v) = F(i - 1, v - A[i]) + C[i]$$

    $$\Rightarrow F(i, v) = Max\{ F(i - 1, v); F(i - 1, v - A[i]) + C[i]) \}$$

- Bài toán nhỏ nhất ứng với i = 0 ta có: F(0, v) = 0

Xây dựng bảng phương án

- Cấu trúc bảng phương án:

– Dùng mảng F[0..n][0..M] chứa giá trị của các F(i, v)

- Cách tính giá trị trên bảng phương án:

  – Điền số 0 cho các ô trên dòng 0

  – Sử dụng công thức đệ qui và giá trị trên dòng (i -1) để tính dòng i

  - Trường hợp A[i] > v:

    F(i, v) = F(i -1, v)

  - Trường hợp A[i] <= v:

    F(i, v) = Max{ F(i -1, v); F(i -1, v – A[i]) + C[i]) }

Ví dụ bảng phương án

- Trường hợp A[i] > v:

  F(i, v) = F(i -1, v)

- Trường hợp A[i] <= v:

  F(i, v) = Max{ F(i -1, v); F(i -1, v – A[i]) + C[i])}

C	A	v / i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	3	1	0	0	0	4	4	4	4	4	4	4	4	4	4	4
5	4	2	0	0	0	4	5	5	5	9	9	9	9	9	9	9
6	5	3	0	0	0	4	5	6	6	9	10	11	11	11	15	15
3	2	4	0	0	3	4	5	7	8	9	10	12	13	14	15	15
1	1	5	0	1	3	4	5	7	8	9	10	12	13	14	15	16

Thuật toán tạo bảng phương án

```
void TaoBangPhuongAn(F[0..n][0..M])
{
        // Điền số 0 cho dòng 0 của bảng
        for (v=0; v <= M; v++) F[0, v] = 0;
        for (i = 1; i <= n; i++)
        for (v=0; v <= M; v++)
        {
                F[i, v] = F[i-1, v];
                If (A[i] <= v && F[i, v] < F[i-1, v - A[i] ] + C[i])
                        F[i, v] = F[ i-1, v - A[i] ] + C[i];
        }
}
```

Truy vết tìm lại các gói hàng đã chọn

Bắt đầu từ ô F[n, M] trên dòng n ta dò ngược về dòng 0 theo nguyên tắc:

- Nếu F[i, v] <> F[i–1, v] thì gói thứ i được chọn, ta truy tiếp ô F[i-1, v-A[i]].
- Nếu F[i, v] = F[i–1, v] thì gói thứ i không được chọn, ta truy tiếp ô F[i-1, v].

C	A	v / i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	3	1	0	0	0	4	4	4	4	4	4	4	4	4	4	4
5	4	2	0	0	0	4	5	5	5	9	9	9	9	9	9	9
6	5	3	0	0	0	4	5	6	6	9	10	11	11	11	15	15
3	2	4	0	0	3	4	5	7	8	9	10	12	13	14	15	15
1	1	5	0	1	3	4	5	7	8	9	10	12	13	14	15	16

Thuật toán truy vết tìm lại các gói hàng đã chọn

```
void TruyVet(F[0..n][0..M])
{
        Bắt đầu từ ô F[n, M] trên dòng n: i = n; v = M;
        for ( ; i > 0; i --)
                if (F[i, v] != F[i-1, v])
                {
                        <Món hàng thứ i được chọn >;
                        v = v – A[i];
                }
}
```